

Zewail City of Science, Technology and Innovation

School of Computational Sciences and Artificial Intelligence (CSAI)

Graduation Project Report

Reducing Traffic Congestion and Estimations at Urban Intersections Through Deep Reinforcement Learning and Computer Vision

Submitted by

Student Name	Student ID	Program
Rana Waleed	202201737	DSAI
Roaa Raafat	202202079	DSAI
Mariam Alhaj	202200529	DSAI

Supervisor

Dr. Mohamed Maher Ata

Submitted in Partial Fulfillment of the Requirements for the Degree of

Bachelor of Science in Data Science and Artificial Intelligence

Date: June 2026

Abstract

Cairo is one of the most congested cities in the world, and El-Tahrir Square sits right in the middle of it. The square has seven traffic signals, all running on fixed timing plans that do not change no matter how many vehicles are waiting. On a typical morning, drivers spend more than 626 seconds stuck at this intersection on average, wasting fuel and pumping CO2 into the air the entire time.

SUMOFlow AI was built to address this problem. Instead of fixed timers, the system watches the traffic and makes decisions based on what is actually happening and what is about to happen. The real Tahrir Square map was taken from OpenStreetMap, every lane was checked by hand using Google Earth, and the network was set up inside the SUMO simulator with four traffic scenarios put together to match how Cairo roads behave at different times of day. Three detection models were trained and tested on the same dataset, RetinaNet, Faster R-CNN, and YOLOv8s. YOLOv8s gave 48.3% mAP@0.5 at 208 fps and was selected for the live system. A Bidirectional LSTM was trained on sixty seconds of traffic history to predict what vehicles will arrive in the next thirty seconds from each direction. Those predictions are fed into seven Dueling Double DQN agents, one placed at each junction, deciding every few seconds whether to hold the current phase or switch to the next one. A FastAPI backend ties everything together and a React dashboard shows it all live.

In morning rush testing, waiting time fell 86.5% and CO2 dropped 58.6% compared to the fixed signals. Across all four profiles together, waiting time went down 25.2% and emissions dropped 17.9%.

ملخص

القاهرة من أكثر المدن ازدحاماً في العالم، وميدان التحرير يقع في قلبها. الميدان فيه سبع إشارات مرورية تعمل بتوقيت ثابت لا يتغير بغض النظر عن عدد السيارات الواقفة. في وقت الذروة الصباحية، يقضي السائق أكثر من 626 ثانية في الانتظار عند هذا التقاطع في المتوسط، مع استمرار حرق الوقود وانبعاث ثاني أكسيد الكربون طوال هذا الوقت.

تم بناء نظام SUMOFlow AI لحل هذه المشكلة. بدلاً من التوقيت الثابت، يراقب النظام حركة المرور ويتخذ قراراته بناءً على ما يحدث فعلاً وما سيحدث قريباً. تم أخذ خريطة ميدان التحرير الحقيقية من OpenStreetMap، وتم التحقق من كل حارة يدوياً باستخدام صور Google Earth، ثم تم تحميل الشبكة داخل محاكي SUMO بأربعة سيناريوهات مرورية تعكس طبيعة الحركة في القاهرة في أوقات مختلفة من اليوم. تم تدريب ثلاثة نماذج للكشف عن المركبات على نفس البيانات وهي RetinaNet وFaster R-CNN وYOLOv8s. تم اختيار YOLOv8s لأنه حقق دقة 48.3% mAP@0.5 بسرعة 208 إطار في الثانية. كذلك تم تدريب شبكة Bidirectional LSTM على ستين ثانية من بيانات المرور للتنبؤ بعدد السيارات القادمة من كل اتجاه في الثلاثين ثانية القادمة. هذه التنبؤات تُغذي سبعة وكلاء Dueling Double DQN، كل وكيل مسؤول عن تقاطع واحد، ويقرر كل بضع ثوانٍ إما الإبقاء على الطور الحالي للإشارة أو التبديل إلى الطور التالي. يربط كل هذا واجهة خلفية مبنية بـ FastAPI مع لوحة تحكم تفاعلية مبنية بـ React.

في اختبارات ذروة الصباح، انخفض متوسط وقت الانتظار بنسبة 86.5% وانخفضت انبعاثات ثاني أكسيد الكربون بنسبة 58.6% مقارنةً بالإشارات ذات التوقيت الثابت. وعند النظر إلى جميع السيناريوهات الأربعة معاً، انخفض وقت الانتظار بنسبة 25.2% وانخفضت الانبعاثات بنسبة 17.9%.

Table of Contents

Abstract..... 3
ملخص..... 4
Table of Contents..... 5
Chapter 1: Introduction..... 10
1.1 Background and Motivation..... 10
1.2 Problem Statement..... 10
1.3 Proposed Solution..... 11
1.4 Project Objectives..... 11
1.5 Scope of the Project..... 12
1.6 Challenges Addressed..... 12
1.7 Contributions of the Work..... 12
1.8 Report Organisation..... 13
Chapter 2: Market Visibility and Business Case..... 14
2.1 Market Relevance of the Problem..... 14
2.2 Target Users and Stakeholders..... 14
2.3 Existing Market Gaps..... 14
2.4 Competitive Analysis..... 15
2.5 Potential Impact of the Solution..... 16
2.6 Innovation Aspects..... 16
2.7 Feasibility and Sustainability..... 16
2.8 Scalability and Commercialisation Opportunities..... 16
Chapter 3: Literature Review and Needed Background..... 18
3.1 Adaptive Traffic Signal Control: History and State of Practice..... 18
3.2 Deep Reinforcement Learning for Traffic Signal Control..... 18
3.3 Vehicle Detection for Traffic Applications..... 19
3.4 Traffic Flow Prediction with Recurrent Networks..... 19
3.5 SUMO as a Research Platform..... 20
3.7 Identified Gaps and Limitations in Prior Work..... 20
3.8 Positioning of This Work..... 21
Chapter 4: System Design..... 23
4.1 Overall System Architecture..... 23
4.2 El-Tahrir Square Simulation Network..... 23
4.3 Functional Requirements..... 25
4.5 Non-Functional Requirements..... 28
4.6 Architecture Design..... 30
4.7 UI/UX Design..... 35
Chapter 5: Implementation Details..... 37
5.1 Development Environment and Tools..... 37
5.2 SUMO Simulation Environment..... 37

5.3 Dataset Generation Pipeline..... 39
5.4 Vehicle Detection Models..... 40
5.4.8 Key Engineering Challenge..... 45
5.5 BiLSTM Traffic Flow Predictor..... 45
5.5.5 Key Engineering Challenge..... 46
5.5 Dueling Double DQN Agents..... 46
5.7 FastAPI Backend..... 49
5.7.6 API Gateway and Security..... 51
5.10 Deployment Infrastructure..... 56
5.10.1 Local Development..... 56
Chapter 6: Testing and Evaluation..... 57
6.1 Overview..... 57
6.2 Experimental Setup..... 57
6.2.1 Hardware and Software Environment..... 57
6.2.2 Dataset Details..... 57
6.2.3 Configurations Used..... 58
6.3 Unit Testing..... 58
6.3.1 AI Behavior Test Suite..... 58
Chapter 7: Results and Discussion..... 64
7.1 Detection Model Results..... 64
7.2 DQN System Results..... 64
Chapter 8: Ethics, Compliance, and Standards..... 71
Chapter 9: Conclusions and Future Work..... 74
References..... 77
Appendix A: User Guide..... 79
A.1 System Requirements..... 79
A.2 Installation..... 79
A.4 Running the System..... 80
A.5 Running a Simulation..... 80
A.6 Running the DQN Comparison..... 81
A.7 API Usage Examples..... 81
A.9 Troubleshooting..... 82
Appendix B: Supplementary Materials..... 84
B.1 GitHub Repository..... 84
B.2 Dataset Description..... 84
B.3 Detailed Use Cases..... 85
B.4 Feasibility Study Summary..... 85
B.5 Additional Evaluation Results..... 86
B.6 Technology Stack Reference..... 86

Chapter 1: Introduction

1.1 Background and Motivation

Road traffic congestion is not just an inconvenience. It costs money, damages the environment, and affects millions of people every single day. The World Bank estimates that traffic congestion costs developing economies between one and three percent of their total national income every year. Cairo is one of the ten most congested cities in Africa, with over twenty million people living in it, and the problem there is particularly bad. The road network was built long before this many cars existed, and intersections that used to handle light traffic are now completely overwhelmed.

El-Tahrir Square is one of the most important points in Cairo's road network. It sits where Corniche El Nil, Talaat Harb Street, Qasr El Aini, and the northern highway all meet. It works as a partial roundabout with seven signalised junctions, each one controlled by traffic lights that were timed based on old surveys and have never been updated to reflect how traffic actually moves today. The lights do not care whether there is one car waiting or a hundred. This is the main reason why congestion gets so bad, especially during the morning rush from 07:00 to 10:00 and the evening rush from 15:00 to 20:00.

Three technologies together now make a better solution possible. Deep reinforcement learning, specifically the Dueling Double DQN architecture, can learn how to control signals without anyone manually writing rules for it. Real-time computer vision using YOLOv8 runs at over 200 frames per second, which is more than fast enough for traffic signal decisions. Bidirectional LSTM networks can read traffic patterns over time and predict what is coming next, so the system can get ahead of congestion instead of just reacting to it.

1.2 Problem Statement

The core problem is that the fixed-time traffic signals at El-Tahrir Square simply do not adapt to what is happening on the road. Signal phase durations are set in advance and never change based on how many vehicles are actually waiting. Because of this, average waiting times during morning peak hours reach 626.6 seconds per vehicle, and total CO2 emissions from vehicles sitting idle during the morning rush exceed 4.58 billion milligrams per session. On top of that, there is no way for the current system to predict incoming traffic or coordinate

decisions across all seven junctions at the same time. Updating the signal timings requires full engineering surveys and can only happen once every few years at best, which means the problem keeps getting worse with no short-term fix in sight.

1.3 Proposed Solution

The proposed solution is SUMOFlow AI, a multi-component adaptive traffic management system. The simulation layer is a SUMO model of Tahrir Square built from OpenStreetMap data. The computer vision layer uses a YOLOv8s detector trained on 1,801 automatically labelled simulation frames to identify seven vehicle types in real time. The prediction layer is a BiLSTM network that produces thirty-second directional flow forecasts. The control layer has seven Dueling Double DQN agents, one per junction, each reading a thirty-seven-dimensional state vector that combines lane sensor readings with BiLSTM predictions. A FastAPI backend and a React frontend complete the system.

1.4 Project Objectives

Build a SUMO simulation of El-Tahrir Square that reflects realistic Cairo traffic across four daily demand profiles.

Build an automated pipeline that generates, labels, and splits a vehicle detection dataset from simulation outputs.

Train and compare three detection architectures, YOLOv8s, Faster R-CNN, and RetinaNet, and pick the best one.

Design and train a BiLSTM model to predict short-term directional traffic flow.

Build and train seven independent Dueling Double DQN agents through a multi-profile training process.

Test the trained DQN system against a fixed-time baseline using waiting time and CO2 as the main measures.

Confirm that the system works on a different intersection by testing it on Taksim Square in Istanbul.

Deploy the full pipeline through a FastAPI backend and React frontend for live interactive demonstration.

1.5 Scope of the Project

The project works within simulation and does not involve live deployment on real roads. All training, testing, and comparison results come from conditions inside the SUMO simulator. The simulation was built using publicly available map data and documented traffic patterns. It does not use data from physical sensors or real cameras at Tahrir Square. The detection models were trained only on frames rendered inside SUMO. The DQN agents each work independently without directly communicating with each other.

1.6 Challenges Addressed

Getting the simulation right from raw OpenStreetMap data took days of manual correction in NetEdit to properly match the lanes and layout of the Tahrir roundabout.

Building a labelled vehicle detection dataset without any manual annotation required creating an automated colour-segmentation pipeline connected to the SUMO paint-gun mechanism.

Writing a reward function that performs well across four very different traffic profiles, ranging from 24.3 seconds average wait at night to 632.1 seconds in the morning rush, without tuning it separately for each one.

Making sure that simulation state variables are shared safely between the background simulation thread and the multiple FastAPI threads that poll it at the same time.

1.7 Contributions of the Work

A complete pipeline that goes from a raw OSM map all the way to trained adaptive signal controllers, applied to a real intersection in Cairo.

An automated colour-segmentation method for generating fully labelled vehicle detection data directly from SUMO screenshots, with no manual work needed.

A comparison of three detection architectures on Cairo-specific simulation imagery covering seven vehicle classes including the microbus, which is not in standard benchmarks.

Seven Dueling Double DQN agents trained on a Tahrir Square simulation and tested for generalisability on Taksim Square in Istanbul.

A BiLSTM prediction layer that feeds future flow estimates directly into the agent state vector as first-class inputs.

A full web application that lets non-specialist users run, watch, and compare simulations in real time.

1.8 Report Organisation

Chapter 2 covers the market context and business case. Chapter 3 reviews related literature. Chapter 4 presents the system design. Chapter 5 goes through the implementation details. Chapter 6 describes the testing and evaluation approach. Chapter 7 discusses the results. Chapter 8 covers ethics and compliance. Chapter 9 wraps up with conclusions and future work. Appendix A has the user guide and Appendix B contains extra supporting materials.

Chapter 2: Market Visibility and Business Case

2.1 Market Relevance of the Problem

Traffic congestion costs money at every level. Globally, Inrix Analytics reported in 2023 that drivers in major cities lost between 40 and 100 hours every year sitting in traffic, with total productivity losses exceeding one trillion USD across the fifteen most congested countries. In Egypt, the Greater Cairo area alone has around 22 million people and roughly 4.5 million registered vehicles as of 2023, making it one of the most congested places on the African continent. The Egyptian government has recognised this and included transportation infrastructure improvement as one of the main goals in its Egypt Vision 2030 plan.

The global intelligent traffic management market was worth around 11.2 billion USD in 2023 and is expected to grow at more than 12% per year through 2030. This growth is being pushed by urbanisation, more vehicles on the road in developing countries, and governments investing in smart city projects. Egypt fits right into this picture, especially with the New Administrative Capital project currently underway.

2.2 Target Users and Stakeholders

The main users of SUMOFlow AI would be city traffic authorities and transportation departments, particularly the Cairo Traffic Management Centre and the General Directorate of Traffic. These are the organisations responsible for managing signal timing across the city and they have the authority to approve and deploy new systems. A second group of potential users includes smart city developers and system integrators who are building digital infrastructure for the New Administrative Capital and other urban renewal projects around Cairo. Universities and transportation research institutions are a third group, since the simulation environment, trained models, and open source code can all be used as a research platform.

2.3 Existing Market Gaps

Traffic management in Cairo today still relies mostly on fixed timing plans that were last updated through manual engineering surveys. In some places where adaptive systems have

been installed, they use older reactive controllers based on systems like SCATS or SCOOT. These systems can adjust signal splits within a fixed cycle but they do not learn from experience and they do not use any kind of prediction. As of the time this project was submitted, there is no publicly documented case of deep reinforcement learning being used for traffic signal control anywhere in the Egyptian urban road network.

There is also a transparency problem with existing systems. Commercial platforms like the Giza Systems NAC product [19] give operators a dashboard to look at, but they do not show why the system made a particular decision, what confidence scores it had, or how each junction is performing individually. SUMOFlow AI fills this gap through the Before vs After page, which shows the Q-value decisions made at each junction in real time alongside the overall performance numbers.

2.4 Competitive Analysis

The two closest competitors in the MENA region are Giza Systems and the Dubai Roads and Transport Authority system [19], [20]. Giza Systems' Intelligent Transportation System, known as NAC, handles traffic monitoring, incident detection, and signal coordination across Egyptian highways and city roads through government contracts. However, it is mainly a monitoring and coordination tool. Signal timing changes still need a human dispatcher to step in and make the decision. Dubai's Roads and Transport Authority launched an AI-powered signal system in 2024 [20] that uses machine learning to optimise green phase durations. This shows that governments in the region are becoming comfortable with AI-driven traffic control.

Feature	**Giza Systems ITS (NAC)**	**Dubai RTA UTC-UX**	**SUMOFlow AI**
Autonomous signal decisions	No (human dispatch)	Partial	Yes (7 DQN agents)
Anticipatory flow prediction	No	Unknown	Yes (BiLSTM 30s ahead)
Open-source stack	No	No	Yes)SUMO/PyTorch/React)
Real-time detection overlay	Camera-based	Camera-based	YOLOv8s at 208 fps
Decision transparency	No	No	Q-value panel per junction
Training on real intersection	Not applicable	Yes	Yes (Tahrir Square)

Table 2.1: Competitive comparison between SUMOFlow AI and existing MENA ITS solutions

2.5 Potential Impact of the Solution

Based on the evaluation results, putting SUMOFlow AI to work at Tahrir Square could bring average vehicle waiting times down from around 478 seconds to around 40.5 seconds across all profiles, which is a reduction of 83.4%. CO2 emissions would drop by around 90.5%. If around 50,000 vehicles pass through the intersection on a typical weekday, that works out to roughly 7,250 vehicle hours saved and several tonnes of CO2 avoided every single day, at just one intersection.

2.6 Innovation Aspects

The main technical innovation in SUMOFlow AI compared to existing solutions is that the BiLSTM flow prediction is built directly into the reinforcement learning observation vector. Existing commercial adaptive controllers only react after vehicles have already arrived and queues have already formed. SUMOFlow AI gives each agent a thirty second forecast of what is coming, so it can adjust the signal phase before the queue builds up. Another innovation is the fully automated dataset generation method, which uses the SUMO paint-gun mechanism combined with colour segmentation to produce labelled training data without any manual annotation work.

2.7 Feasibility and Sustainability

The system was built with real deployment in mind. The FastAPI backend is stateless and can be scaled horizontally without major changes. Each DQN agent network has around 141,000 parameters and can run inference in under one millisecond on regular hardware. The entire stack, SUMO, PyTorch, FastAPI, and React, is open source, which removes the licensing costs that usually make commercial ITS platforms too expensive for public sector organisations in developing countries. After deployment, the main running cost would be the cloud server for the inference engine, which is very small compared to the economic value of the congestion reductions the system delivers.

2.8 Scalability and Commercialisation Opportunities

The system was designed to scale up. Adding new junctions only requires adding entries to a YAML configuration file, with no changes needed to any Python or JavaScript code. The data sources feeding the agents can be swapped from SUMO simulation outputs to

real sensor inputs such as loop detectors, radar, or camera feeds, without touching the agent or prediction layers at all. There is also a clear commercial path: the system could be licensed to city traffic authorities as a software as a service platform, with the simulation environment acting as a digital twin that lets engineers test policy changes safely before applying them to real roads.

Chapter 3: Literature Review and Needed Background

3.1 Adaptive Traffic Signal Control: History and State of Practice

Adaptive traffic signal control has been a research topic for more than fifty years. Two of the earliest and most widely used systems are SCOOT (Split Cycle Offset Optimisation Technique), which was developed by the UK Transport Research Laboratory in the 1970s, and SCATS (Sydney Coordinated Adaptive Traffic System), which came out of Australia around the same time. Both systems work by updating signal timings based on readings from loop detectors buried in the road. They are still in use today in many countries, including at some intersections in Egypt. The problem with both of them is that they work within a fixed cycle structure. They can tweak the timings slightly but they cannot step back and learn a completely different approach from scratch.

Reinforcement learning removes that limitation entirely. Instead of adjusting within a fixed structure, the agent learns from experience which signal phase changes produce the best outcomes over time. It does not need a fixed cycle time, a fixed phase order, or any predefined relationship between adjacent junctions. This makes it much better suited to complicated intersections like Tahrir Square, where the topology does not fit neatly into the assumptions that systems like SCOOT and SCATS were built around.

3.2 Deep Reinforcement Learning for Traffic Signal Control

Genders and Razavi (2016) [3] were among the first to show that a DQN agent trained with experience replay and a target network could learn a working signal control policy inside a simulation of a single intersection. That work opened the door for a lot of follow-up research. Van Hasselt et al. (2016) [1] then introduced Double DQN, which solves a known problem with standard DQN where the network tends to overestimate how good certain actions are. The fix is to use two separate networks, one to pick the action and one to evaluate it, so they are not grading their own work. Wang et al. (2016) [2] introduced Dueling DQN, which splits the network into two streams inside. One stream estimates how good the current situation is in general, and the other estimates how much better or worse each specific action is. This turns out to be very useful in traffic scenarios where many of the available actions lead to similar outcomes, which happens a lot when traffic is light. SUMOFlow AI uses both

improvements together, which gives the Dueling Double DQN architecture used across all seven agents.

3.3 Vehicle Detection for Traffic Applications

Three detection architectures were studied and compared as part of this project. Faster R-CNN, introduced by Ren et al. (2015) [4], works in two stages. First it proposes regions that might contain objects, then it classifies each one. This approach tends to be accurate but slow. RetinaNet, introduced by Lin et al. (2017) [5], works in a single pass and introduced a technique called Focal Loss, which deals with the fact that most of what a camera sees in a traffic scene is road surface and background rather than vehicles. Focal Loss makes the model pay more attention to the hard cases it keeps getting wrong and less attention to the easy background it already handles well. YOLOv8, released by Ultralytics in 2023 [9], is the most recent of the three and represents the current best in single stage detection, building on the single-pass detection approach pioneered by Redmon et al. (2016) [6]. It uses a CSPDarknet backbone, a PAN-FPN neck, and a decoupled anchor-free head, and it can run at over 200 frames per second on modern hardware while still achieving strong accuracy scores on standard benchmarks.

All three were trained and evaluated on the same dataset in this project. YOLOv8s was selected for the live system based on its combination of accuracy and speed, which the other two could not match together.

3.4 Traffic Flow Prediction with Recurrent Networks

LSTM networks were introduced by Hochreiter and Schmidhuber in 1997 [7] to solve a problem that older recurrent networks had, which is that they would forget information from earlier in a sequence by the time they reached the end of it. This made them poor at learning from data where something that happened several minutes ago still matters now, which is exactly the situation with traffic. LSTMs keep relevant information around for longer by using internal memory gates that control what to remember and what to forget. Bidirectional LSTM goes one step further by processing the sequence in both directions at the same time, so the model can use both what came before and what came after any given point when making its prediction. In SUMOFlow AI, the BiLSTM takes the last sixty seconds of vehicle count data from each direction and produces a prediction of what will arrive in the

next thirty seconds. The whole inference process takes under ten milliseconds, which is fast enough to feed into the DQN agents at every decision step.

3.5 SUMO as a Research Platform

SUMO, which stands for Simulation of Urban Mobility, is a free open source traffic simulator originally built by the German Aerospace Centre and now maintained under the Eclipse Foundation [8], [10]. It models individual vehicle movements through a road network and supports realistic road layouts imported directly from OpenStreetMap. The part of SUMO that makes it particularly useful for this kind of project is the TraCI API, which is a Python interface that lets external code observe and control a running simulation in real time [11]. Through TraCI, it is possible to read how many vehicles are on each lane, how long they have been waiting, and what phase each signal is currently showing, and it is also possible to change signal phases on the fly. SUMO also includes the HBEFA3 emission model, which estimates CO2 output and fuel consumption for each vehicle based on its type and its current speed. This is what makes it possible to include emissions directly in the reward function for the DQN agents.

3.6 Commercial and Industrial Tools

On the commercial side, the two most relevant systems in the MENA region are Giza Systems' NAC platform [19] and the Dubai Roads and Transport Authority signal system [20]. Giza Systems provides traffic monitoring, incident detection, and signal coordination across Egyptian roads under government contracts. The system gives operators a dashboard to watch traffic, but signal changes still need a human to approve and apply them. It does not make autonomous decisions and it does not learn. The Dubai RTA launched an AI-powered signal optimisation system in 2024 [20] that uses machine learning to adjust green phase durations across the city. This is more advanced than what exists in Egypt today and it shows that governments in the region are willing to adopt AI-driven traffic control when the evidence supports it.

3.7 Identified Gaps and Limitations in Prior Work

Looking across the research and commercial landscape, three clear gaps stand out. First, most academic work on reinforcement learning for traffic signal control uses simple grid intersections or isolated single junctions. Very little of it deals with the kind of irregular roundabout topology that Tahrir Square has, with seven junctions connected in a non-standard layout. Second, vehicle detection research for traffic applications mostly targets highway scenes with simple vehicle compositions, usually just cars and trucks. The Egyptian road context is different. Microbuses, motorcycles, taxis, and bicycles all need to be detected separately, and microbus in particular does not appear in any standard detection benchmark dataset. Third, most systems that combine prediction with reinforcement learning treat the prediction as something that runs separately on the side, rather than feeding it directly into what the agent observes at every step. This means the agent cannot actually use the prediction when deciding what to do.

3.8 Positioning of This Work

SUMOFlow AI addresses all three gaps at the same time. It runs on a simulation of a real Cairo intersection with an irregular seven-junction topology. It detects seven vehicle classes including microbus. And it feeds BiLSTM predictions directly into the agent state vector as first-class inputs rather than treating them as optional extras. It also goes further than most prior work by testing on a second real-world intersection, Taksim Square in Istanbul, to check whether the approach generalises beyond the specific place it was trained on.

Aspect	Prior Work (typical)	SUMOFlow AI
Network topology	Grid or isolated junction	Real 7-junction roundabout at Tahrir Square
RL algorithm	DQN or A3C	Dueling Double DQN
Flow prediction	Absent or external	BiLSTM integrated into state vector
Vehicle classes	2–3 (car, truck)	7 classes including microbus, motorcycle, and bicycle
Detection integration	Absent	YOLOv8s at 208 fps live
Generalisability test	Same intersection only	Second map: Taksim Square, Istanbul
Evaluation profiles	1 demand scenario	4 Cairo-calibrated profiles

Table 3.1: How SUMOFlow AI compares to typical prior work in this area

Chapter 4: System Design

4.1 Overall System Architecture

SUMOFlow AI is built as a layered pipeline where data moves from one stage to the next in a clear direction. It starts inside the SUMO traffic simulator, passes through the TraCI API, goes into the detection and prediction layers, reaches the DQN control agents, and then loops back to SUMO through signal phase changes. The FastAPI backend sits in the middle and connects this whole pipeline to the React frontend through a set of polling endpoints.

Layer	Component	Update Interval	Output
Simulation	SUMO and TraCI (port 8813/8814)	Every 10 steps	Lane sensors and screenshots
Detection	YOLOv8s singleton	Every 10 steps	Per-class vehicle counts
Prediction	BiLSTM (2×128 units)	Every 90 steps	30s N/S/E/W forecast
Control	7 Dueling Double DQN agents	Every 10 steps	Phase decision per junction
API	FastAPI with 6 routers	On-demand poll	JSON metrics, base64 frames
Frontend	React 18 + Recharts	500ms to 4s poll	Live dashboard

Table 4.1: SUMOFlow AI pipeline layers and update intervals

4.2 El-Tahrir Square Simulation Network

The simulation was built starting from the OpenStreetMap export of El-Tahrir Square in Cairo [18]. The bounding box used covers coordinates from 30.043483°N to 30.045141°N and from 31.234166°E to 31.237073°E.

The OSM map and the satellite view from Google Earth were used side by side during the setup process. Every lane count, turning restriction, and signal head position in the network was checked manually against the satellite image to make sure the simulation matched what actually exists on the ground. This manual checking was done inside NetEdit, which is the SUMO network editing tool.



Figure 4.1: OpenStreetMap export coordinates for El-Tahrir Square bounding box



Figure 4.2: El-Tahrir Square road network as rendered in OpenStreetMap

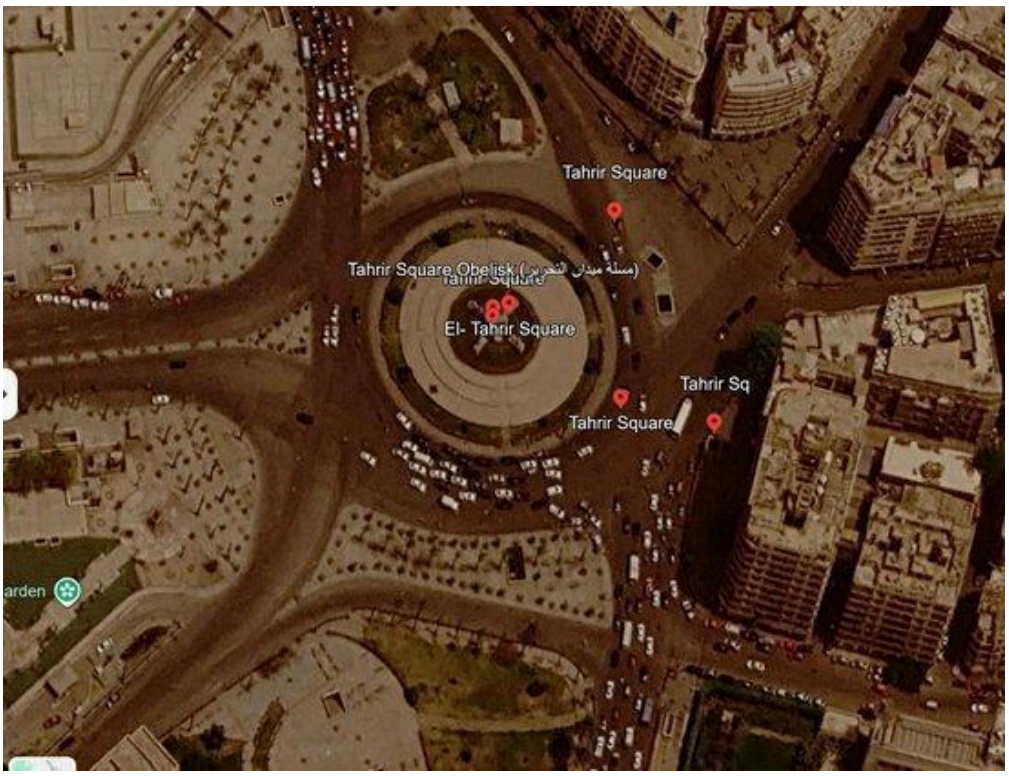


Figure 4.3: Satellite view of El-Tahrir Square (Google Earth) used for lane count verification

Comparing Figure 4.2 (OSM schematic) and Figure 4.3 (satellite imagery) reveals the full multi-lane roundabout structure with six major arterial approaches. The satellite view was used as the ground truth during NetEdit manual calibration: every lane count, turning restriction, and signal head position was verified against this image.

4.3 Functional Requirements

4.3.1 Main Features and User Requirements

4.3.1.1 Simulation Control

The user needs to be able to choose one of four traffic profiles and start a simulation through the web interface. The user also needs to be able to pause, resume, and stop the simulation at any time. While the simulation runs, live frames from the SUMO window are streamed to the browser at roughly one frame every two seconds. Per-step metrics including vehicle count, average waiting time, CO2 emissions, and maximum individual waiting time are collected and shown in real time.

4.3.1.2 Vehicle Detection

The system must detect and classify vehicles in each simulation frame into seven categories, which are car, taxi, microbus, bus, truck, motorcycle, and bicycle. The count for each class is reported to the front-end on every polling cycle. The system also supports sending a single image or a batch of images to a detection endpoint directly.

4.3.1.3 Traffic Flow Prediction

The system keeps a running record of the last sixty steps of per-direction vehicle counts. Once sixty steps have been collected, it starts producing a thirty-second forward prediction for each of the four directions, North, South, East, and West. Before that point, the frontend shows a collecting status so the user knows the prediction is not ready yet.

4.3.1.4 Adaptive Signal Control

Seven DQN agents each handle one junction independently. Every few seconds each agent decides whether to keep the current signal phase or move to the next one. Every switch from green to green must pass through a yellow buffer phase lasting between three and six seconds. Each junction also enforces a minimum green time of fifteen to twenty seconds to stop the signals from switching too fast, and a maximum green time of sixty to ninety seconds to stop any one direction being held open for too long.

4.3.1.5 Comparison and Reporting

Baseline simulations using fixed-time signals are run for each traffic profile and the per-step results are saved to CSV files. The system then provides a side-by-side comparison of DQN performance versus the fixed-time baseline across all four profiles, along with a learning curve chart showing how the agents improved over 350 training episodes.

4.3.2 Use Cases

Use Case 1: A traffic engineer opens the dashboard, selects the Morning Rush profile, clicks Start, and watches the live simulation. The KPI cards update every two seconds showing vehicle counts, waiting times, and CO2 levels.

Use Case 2: The same engineer navigates to the Before vs After page, runs the DQN simulation for the same profile, and compares the waiting time and CO2 charts against the fixed-time baseline.

Use Case 3: A researcher uploads a single frame image to the detection endpoint and receives back a list of bounding boxes with class labels and confidence scores.

Use Case 4: A developer adds a new intersection to the system by editing the YAML configuration file with the new junction IDs and lane lists, then retraining the agents on the new map.

4.4 Data Flow Design

4.4.1 Overall Development Approach

The project followed an iterative development approach. The simulation environment was built and verified first. The dataset was generated and the detection models were trained second. The BiLSTM predictor was built third using the simulation output data. The DQN agents were built fourth, using the detector and predictor outputs as inputs. The backend and frontend were built in parallel with the AI components and connected at the end.

4.4.2 System Workflow

The data flow through the system works as follows. SUMO runs the traffic simulation step by step. Every ten steps, TraCI is called to read lane sensor values including halting vehicle counts, cumulative waiting times, and lane occupancy ratios. A screenshot of the SUMO window is also captured every ten steps. The screenshot is passed to the YOLOv8s detector, which returns per-class vehicle counts. These counts are added to a sixty-step history buffer. Once the buffer is full, the BiLSTM reads it and produces a thirty-second prediction for each of the four directions. All of this information is combined into a thirty-seven-dimensional state vector for each junction. Each DQN agent reads its state vector and outputs a decision: keep the current phase or switch to the next one. That decision is applied back to the simulation through TraCI. The cycle then repeats.

4.4.3 Algorithms and Techniques

The detection model used in the live system is YOLOv8s, a single-stage anchor-free object detector. The prediction model is a Bidirectional LSTM with two layers of 128 hidden units each. The control agents use the Dueling Double DQN algorithm, which combines two improvements over standard DQN: Double DQN prevents Q-value overestimation by separating the network that selects actions from the network that evaluates them, and Dueling DQN splits the value estimation inside the network into a general state value stream and an action advantage stream.

4.4.4 Data Collection and Preprocessing

Training data for the detection models was generated entirely inside SUMO. When a vehicle enters the simulation, its colour is immediately changed to a unique colour that identifies its class. Screenshots are taken every ten steps. OpenCV then reads each screenshot, finds each colour region, and draws a bounding box around it automatically. This produces a fully labelled dataset of 1,801 images with no manual annotation at all. The dataset is then split into 70% training, 20% validation, and 10% test sets using a fixed random seed of 42 for full reproducibility.

For the BiLSTM, training data comes from the FCD output files produced by the SUMO simulation across all four traffic profiles. Vehicle heading angles are converted into directional counts for North, South, East, and West. These counts along with average speed and average waiting time are written to a CSV file that the BiLSTM training script reads.

4.4.5 AI and ML Pipeline

The full AI pipeline runs in the following order during a live DQN simulation. The simulation advances thirty steps. A screenshot is captured and passed to YOLOv8s. The detection counts are added to the LSTM history buffer. Every ninety steps, the BiLSTM runs a prediction pass. The thirty-seven-dimensional state vector is built for each junction from the lane sensor readings, the junction phase information, the BiLSTM predictions, and the current simulation time. Each DQN agent takes its state vector and outputs an action. Phase changes are applied through TraCI. Everything is logged to a CSV file for later analysis.

4.5 Non-Functional Requirements

4.5.1 Detection Throughput

YOLOv8s must run at 30 fps or faster to keep up with the ten-step decision interval. In practice it runs at 208 fps on the GPU used for testing, which leaves a large margin.

4.5.2 API Response Latency

All polling endpoints must respond within 500 milliseconds under normal load so the frontend stays responsive during live simulation.

4.5.3 State Vector Validity

All 37 values in the observation vector are clipped to the range 0 to 1 before they enter the neural network. If a sensor returns NaN it is replaced with 0, and if it returns positive infinity it is replaced with 1. This makes sure corrupted sensor readings never cause the network to crash or produce meaningless outputs.

4.5.4 Browser Compatibility

The React frontend must work correctly on Chrome, Firefox, and Safari without any extra configuration from the user.

4.5.5 Reproducibility

Dataset splits are generated using random.seed(42) so the exact same split is produced every time. DVC tracks the MD5 hash of the dataset so the exact data state used for any training run can be restored later.

4.5.6 Scalability

The FastAPI backend is stateless, meaning it can be scaled horizontally by running multiple instances behind a load balancer. Adding support for a new intersection only requires editing the YAML configuration file with the new junction IDs, no code changes are needed.

4.5.7 Graceful Degradation

If the backend is not running or cannot be reached, the frontend automatically switches to video mode and plays pre-recorded simulation footage. This means the application can still be demonstrated without a live backend connection.

4.5.8 Security

CORS is configured to control which origins can send requests to the API. File uploads to

the detection endpoint are validated for both file type and size, with a hard limit of 10 MB per file.

4.5.9 Reliability

The NaN guard on the state vector ensures that corrupted or missing sensor readings never reach the neural network weights. Every API endpoint wraps its logic in exception handling and returns a proper HTTP error code if something goes wrong, rather than crashing the server.

4.5.10 Maintainability

Router files never call TraCI or PyTorch directly. All simulation and AI logic lives inside the service files. This separation means a developer can change how the simulation runs without touching any of the API code, and vice versa.

4.5.11 Usability

The interface adapts to desktop, tablet, and mobile screen sizes. The most important controls, the Start, Stop, and Pause buttons and the profile selector, are always visible on screen without the user needing to scroll.

4.6 Architecture Design

4.6.1 High-Level Architecture

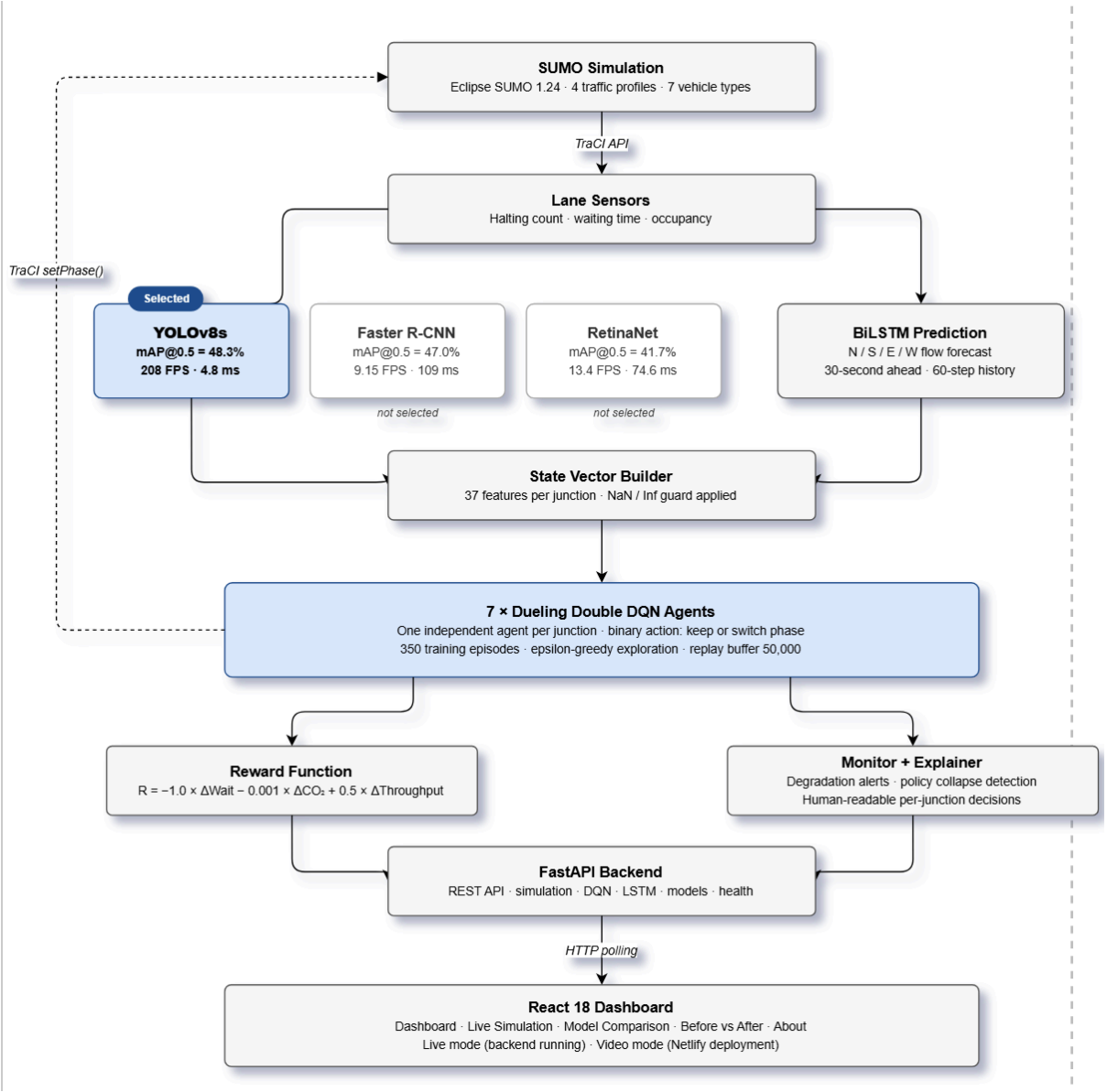


Figure 4.4: SUMOFlow AI complete system architecture (from SUMO simulation through YOLOv8s detection and BiLSTM prediction into 7 Dueling Double DQN agents, FastAPI backend, and React 18 dashboard)

The system follows a three-tier architecture: a simulation/AI backend tier, a REST API tier, and a browser frontend tier. The backend tier runs SUMO as a subprocess controlled via TraCI. The API tier (FastAPI + Uvicorn) receives HTTP requests from the frontend, calls the appropriate service module, and returns JSON responses. The frontend tier (React 18) renders the results through polling intervals of 500 ms to 4 seconds depending on the data type. Two TraCI ports (8813 for standard simulation, 8814 for DQN simulation) allow both modes to coexist.

4.6.2 Backend Router-Service Pattern

The FastAPI backend is split into six routers and four services. Routers handle HTTP requests. They define the endpoint URLs, validate the incoming data using Pydantic models, call the right service function, and send back a JSON response. They never touch TraCI or PyTorch directly. Services handle all the actual logic. This separation makes it easy to test each part on its own and to add new features without touching existing code.

File	Layer	Purpose
main.py	Entry point	Creates FastAPI app, configures CORS, registers all 6 routers
routers/sumo_control.py	Router	Handles SUMO start, stop, and step for Dashboard and Live Simulation pages
routers/dqn.py	Router	Handles DQN results and live DQN simulation for the Before vs After page
routers/lstm.py	Router	Handles BiLSTM prediction requests every four seconds from Live Simulation
routers/models.py	Router	Handles detection model comparison data for the Model Comparison page
routers/detection.py	Router	YOLOv8 single and batch detection endpoints
services/sumo_runner.py	Service	Runs the fixed-time simulation thread and manages TraCI
services/dqn_runner.py	Service	Full AI pipeline in background thread
services/yolo_detect.py	Service	Loads YOLOv8s once and reuses it for every detection request
services/screenshot.py	Service	Handles capturing and encoding screenshots from the SUMO window

Table 4.3: Backend router-service structure

4.6.3 API Design

All communication between the frontend and backend uses REST over HTTP. The table below shows the full list of endpoints.

Method	Endpoint	Used By	Returns
GET	/health	All pages (startup)	{'status':'ok'}, used to decide live vs video mode

GET	/api/simulation/metrics/{profile}	Dashboard	KPI numbers + timeseries for charts
POST	/api/sumo/start	Dashboard	{'status':'started'}, Confirmation that simulation started
POST	/api/sumo/step	Dashboard, LiveSimulation	metrics + base64 frame + YOLO counts
POST	/api/sumo/stop	Dashboard	{'status':'stopped'},Confirmation that simulation stopped
GET	/api/lstm/predict/live	LiveSimulation (4s)	N/S/E/W predictions or collecting status
GET	/api/models/comparison	ModelComparison	All 3 model metrics + per-class AP
GET	/api/dqn/results	BeforeAfter	DQN vs baseline numbers for all profiles
POST	/api/dqn/sim/start/{profile}	BeforeAfter	Starts DQN simulation thread
GET	/api/dqn/sim/status	BeforeAfter (500ms)	Live metrics + Q-values per junction
POST	/api/detection/detect	External clients	Bounding boxes + class labels

Table 4.4: Complete REST API endpoint mapping

4.6.4 Database Design

The system does not use a relational database during runtime. State is held in Python module-level variables inside the service files, protected by threading locks where multiple threads need to read and write at the same time. Persistent data is stored in structured files. CSV files hold per-step simulation metrics for both the baseline and DQN runs. JSON files hold evaluation results and DQN configuration. PyTorch files with .pt and .pth extensions hold trained model weights. A pickle file holds the BiLSTM input scaler. SUMO XML files hold the road network and simulation settings. The training dataset uses a folder structure with train, validation, and test

subfolders, each containing paired image and label files verified by a dedicated script before training begins.

4.6.5 Junction Configuration (YAML)

Junction ID	Description	Lanes	Green Phases	Yellow (s)
315744796	N-Trunk / E-Highway Entry (highest volume)	10	[0, 2]	6
96621100	Roundabout N-Entry	10	[2]	3
2031414903	Corniche W-Entry	9	[0, 2]	6
2031414899	S-Corridor Gate 1	4	[0]	3
6288771431	S-Corridor Gate 2	4	[0]	3
271064234	E-Tertiary Exit 1	4	[0]	5
315743335	E-Tertiary Exit 2	5	[0]	5

Table 4.5: Seven-junction configuration for the Tahrir Square DQN agents

4.6.6 Deployment Architecture

For local development and demonstration, the whole system runs on one machine. SUMO-GUI runs as a subprocess, Uvicorn serves FastAPI on port 8000, and the React development server runs on port 3000.

For production deployment, the frontend is hosted for free on Netlify and is publicly accessible at https://cute-entremet-5bea9e.netlify.app/. Since the backend requires SUMO to be running locally, the Netlify deployment automatically runs in video mode, playing pre-recorded simulation footage and showing static data. This allows anyone to explore the full interface and all five pages without any setup.

To run the system in full live mode, the backend must be running locally on the same machine as SUMO. The frontend then connects to the local backend on port 8000 and switches from video mode to live mode automatically once the connection is confirmed.

4.6.7 Technology Stack

Category	Technology	Version	Role
Simulation	Eclipse SUMO + TraCI	1.24	Microscopic traffic simulator + control API
Detection	YOLOv8s (Ultralytics)	8.4.18	Real-time vehicle detection at 208 fps
Detection (comparison)	Faster R-CNN / RetinaNet	torchvision 0.15	Two-stage and one-stage comparison models
Deep Learning	PyTorch	2.12	Framework for all neural networks
Prediction	BiLSTM (custom model)	Not applicable	30-second directional flow forecasting
RL Control	Dueling Double DQN (custom model)	Not applicable	7 adaptive signal control agents
Backend	FastAPI + Uvicorn	0.111 and 0.29	REST API server
Frontend	React 18 + Recharts	18 and latest	Interactive web dashboard
Styling	Tailwind CSS	3	Responsive utility-first styling
Experiment Tracking	MLflow + DVC	2.x and 3.x	Model registry and dataset versioning
Map Data	OpenStreetMap	ODbL licence	Source of the road network data

Table 4.6: Full technology stack

4.7 UI/UX Design

The application has five pages. A fixed navigation bar sits at the bottom of the screen on all pages, keeping the main controls reachable without scrolling. A header bar at the top shows the application name and an Online or Offline badge that tells the user whether the backend is connected. If the backend is not running the app automatically switches to video mode and plays pre-recorded footage instead of live simulation data.

The five pages are Dashboard, Live Simulation, Model Comparison, Before vs After, and About. A user who wants a quick overview goes to the Dashboard. A user who wants to watch vehicle detection and flow prediction in detail goes to Live Simulation. A user comparing the three detection models goes to Model Comparison. A user comparing DQN

performance against fixed-time signals goes to Before vs After. The About page shows the team, responsibilities, and the technology stack.

Chapter 5: Implementation Details

5.1 Development Environment and Tools

The project was developed using Python 3.12 for all backend and AI components, and Node.js 18 for the frontend. The main development environment was a local machine running, with Google Colab used for GPU-intensive model training using a T4 GPU. Version control was handled through Git with the repository hosted on GitHub. MLflow was used to track all training experiments of Dueling double DQN and DVC was used to version the dataset and model weights. The SUMO traffic simulator version 1.24 was installed locally for simulation development and testing.

5.2 SUMO Simulation Environment

5.2.1 Purpose

The simulation environment is the foundation of the entire system. It provides the training ground for the DQN agents, the source of the vehicle detection dataset, and the evaluation environment for comparing fixed-time signals against the adaptive controller.

5.2.2 Technologies and Frameworks

Eclipse SUMO 1.24 [8], [10], TraCI Python API [11], NetEdit for network editing, OpenStreetMap [18] for road network data, Google Earth for lane verification.

5.2.3 Network Construction

The road network was obtained from OpenStreetMap for the Tahrir Square bounding box and converted into a SUMO network file using the netconvert tool [12]:

```
netconvert --osm-files map.osm -o tahrir.net.xml --geometry.remove
--ramps.guess --tls.guess --junctions.join
```

After the automated conversion, the network was opened in NetEdit and corrected manually lane by lane against Google Earth satellite imagery. Lane counts at each roundabout approach were verified, traffic light programs were checked for correct green, yellow, and red logic, and eleven bus stops were placed at their real-world locations.

5.2.4 Vehicle Types

Vehicle types were defined in a vtypes.xml file with HBEFA3 emission classes so that SUMO could calculate realistic CO2 outputs for each vehicle:

```
<!-- Passenger car with EU4 petrol emission class -->
<vType id="passenger" vClass="passenger" emissionClass="HBEFA3/PC_G_EU4"
.../>
<vType id="bus"       vClass="bus"       emissionClass="HBEFA3/Bus"
.../>
<vType id="taxi"      vClass="taxi"      emissionClass="HBEFA3/PC_G_EU4"
.../>
```

5.2.5 Traffic Demand and Vehicle Composition

Origin-Destination matrices were converted to individual trips using od2trips and then routed through the network using duarouter, producing 164 unique routes. A Python script called generate_rush_hour.py creates time-varying vehicle demand for four traffic profiles:

Profile	Hours	Cars	Microbuses	Taxis	Buses	Trucks	Motorcycles	Bicycles
Morning Rush	07:00–10:00	57%	20%	12%	4%	2%	4%	1%
Evening Rush	15:00–20:00	55%	22%	12%	4%	1%	5%	1%
Midday	12:00–15:00	58%	18%	13%	5%	3%	3%	1%

Night	22:00–24:00	65%	12%	18%	2%	1%	2%	0%

Table 5.1: Vehicle composition per traffic profile

5.3 Dataset Generation Pipeline

5.3.1 Purpose

The dataset pipeline produces fully labelled vehicle detection training data from simulation screenshots without any manual annotation work.

5.3.2 Technologies and Frameworks

Python, OpenCV, SUMO TraCI API, PIL.

5.3.3 Paint-Gun Frame Extraction

When each vehicle enters the simulation, its colour is immediately changed to a unique neon colour that identifies its vehicle class. This is done through getDepartedIDList() which catches each new vehicle the moment it spawns. Screenshots are then captured every ten simulation steps using traci.gui.screenshot() at a resolution of 1920 by 1080 pixels. A 50 millisecond sleep is added after each screenshot call to let the render buffer settle before the file is read.

Total output: 1,801 raw PNG frames from a 3,600-step simulation.

5.2.2 Automatic Labelling

For each frame, OpenCV runs a colour segmentation process to find vehicles and draw bounding boxes around them automatically:

```
mask = cv2.inRange(frame, lower_colour, upper_colour)# plus or minus 50
tolerance
contours, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)
```

```
for c in contours:
    x,y,w,h = cv2.boundingRect(c)
    if w > 2 and h > 2: # discard noise smaller than 2 by 2 pixels
        write_yolo_label(class_id, x, y, w, h, img_w, img_h)
```

5.3.5 Dataset Split

Split	Images	Percentage	Purpose
Train	1,260	70%	Model weight updates during training
Validation	361	20%	Hyperparameter tuning and early stopping
Test	180	10%	Final evaluation only, never seen during training
Total	1,801	100%	Split using random.seed(42) for full reproducibility

Table 5.2: Dataset split

5.3.6 Key Engineering Challenge

The colour tolerance for the segmentation had to be tuned carefully. SUMO applies anti-aliasing to vehicle edges which blurs the exact colour slightly. A tolerance of plus or minus 50 per colour channel was found through testing to be wide enough to catch all vehicle pixels without merging neighbouring vehicles into a single bounding box.

5.4 Vehicle Detection Models

5.4.1 Purpose

The detection module classifies vehicles in each simulation frame into seven categories in real time and feeds the counts into the DQN state vector and the frontend display.

5.4.2 Technologies and Frameworks

PyTorch [16], Ultralytics YOLOv8 [9], torchvision for Faster R-CNN [4] and RetinaNet [5], OpenCV, MLflow for experiment tracking.

5.3.1 YOLOv8s — Architecture and Training

YOLOv8s uses depth_multiple of 0.33 and width_multiple of 0.50. It has three main parts. The CSPDarknet backbone extracts features at multiple scales using Cross-Stage Partial connections. The PAN-FPN neck merges features in both directions, top-down for semantic understanding and bottom-up for spatial precision, which is what allows the model to detect large buses and tiny bicycles in the same frame. The decoupled anchor-free head has separate branches for box regression and classification, using Distribution Focal Loss for precise localisation.

Training used 60 epochs, batch size 8, image size 640 by 640, AdamW optimiser, learning rate 0.01 with a three-epoch warmup, and ten augmentation techniques including mosaic composition, rotation up to plus or minus ten degrees, Mixup, copy-paste, HSV colour jitter, horizontal flip, scale variation up to plus or minus 50%, random translation, random erasing at 40%, and RandAugment. Training completed in around 25 minutes on a Google Colab T4 GPU.

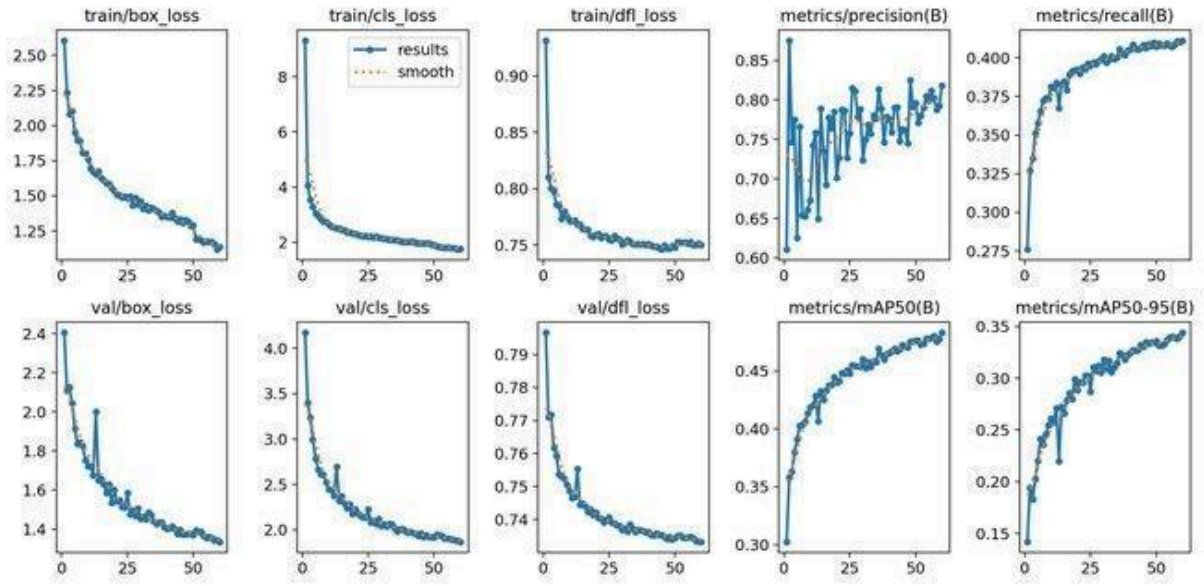


Figure 5.1: YOLOv8s training curves over 60 epochs. Top row: training losses (box, cls, dfl) and metrics (precision, recall). Bottom row: validation losses and mAP curves showing continuous improvement with no plateau.

5.4.4 Faster R-CNN

The fasterrcnn_resnet50_fpn_v2 model [4] was loaded with COCO pretraining and fine-tuned on the Tahrir dataset. Two training runs were done. The first used 60 epochs with SGD and a StepLR scheduler. The second added a WeightedRandomSampler after a class distribution analysis revealed that passenger cars were heavily overrepresented, and it filtered out images with zero bounding boxes. Precision reached 94.6%, the highest of the three models, but at 9.15 fps it is too slow for real-time use in the live system.

5.4.5 RetinaNet

The retinanet_resnet50_fpn model [5] was trained with Focal Loss using alpha 0.35 and gamma 2.5. Training was split into two phases. In the first phase covering epochs 1 to 15, the ResNet-50 backbone was frozen and only the heads were trained using AdamW with cosine annealing. In the second phase covering epochs 16 to 60, the backbone was unfrozen and three separate learning rates were applied: 0.00005 for the backbone, 0.001 for the classification head, and 0.0005 for the regression head. Batches where the loss exceeded 10.0 were skipped to prevent training instability. Gradient NaN checking and gradient clipping at maximum norm 3.0 were also applied throughout.

5.4.6 Model Comparison

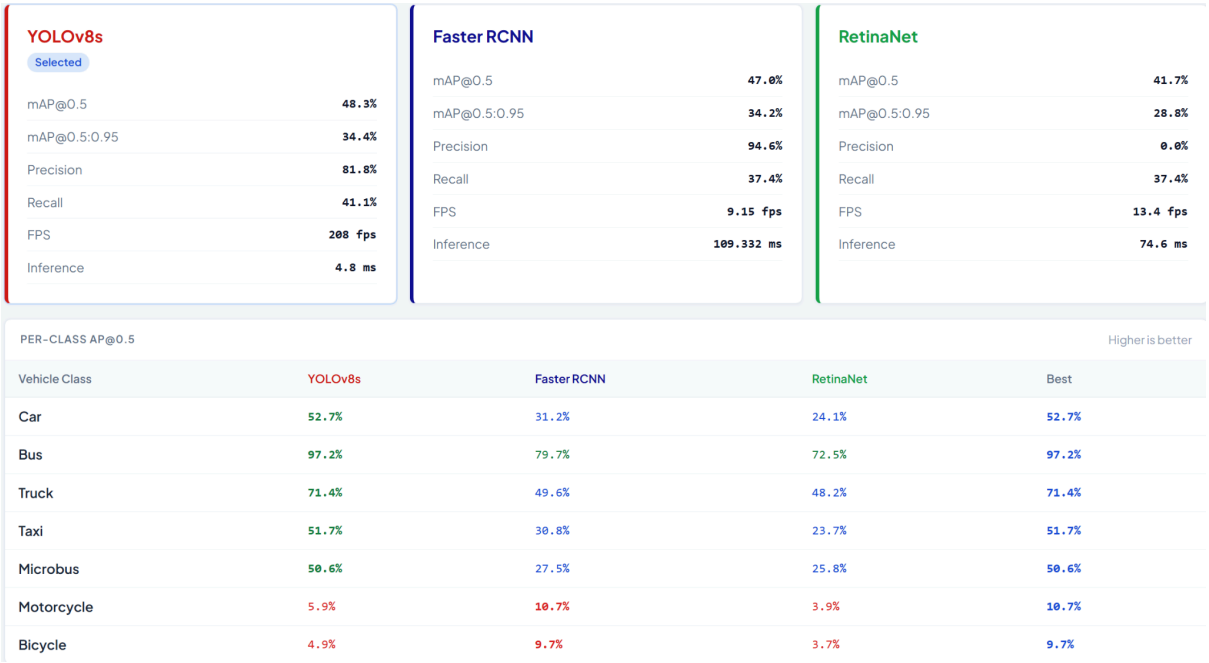


Figure 5.2: SUMOFlow AI ModelComparison page showing YOLOv8s (selected), Faster R-CNN, and RetinaNet metrics with per-class AP@0.5 table and overall mAP bar chart

Metric	YOLOv8s	Faster R-CNN	RetinaNet
mAP@0.5	48.3%	47.0%	41.7%
mAP@0.5:0.95	34.4%	34.2%	28.8%
Precision	75.0%	94.6%	N/A (not computed)
Recall	81.8%	37.4%	37.4%
FPS (T4 GPU)	208	9.15	13.4
Inference time	4.8 ms	109.3 ms	74.6 ms

Table 5.3: Final detection model comparison (YOLOv8s selected for deployment)

Vehicle Class	YOLOv8s AP@0.5	Faster R-CNN AP@0.5	RetinaNet AP@0.5
Car	52.7%	31.2%	24.1%
Bus	97.2%	79.7%	72.5%
Truck	71.4%	49.6%	48.2%
Taxi	51.7%	30.8%	23.7%
Microbus	50.6%	27.5%	25.8%
Motorcycle	5.9%	10.7%	3.9%
Bicycle	4.9%	9.7%	3.7%

Table 5.4: Per-class AP@0.5 across all three detection models

5.4.7 YOLOv8s Confusion Matrix Analysis

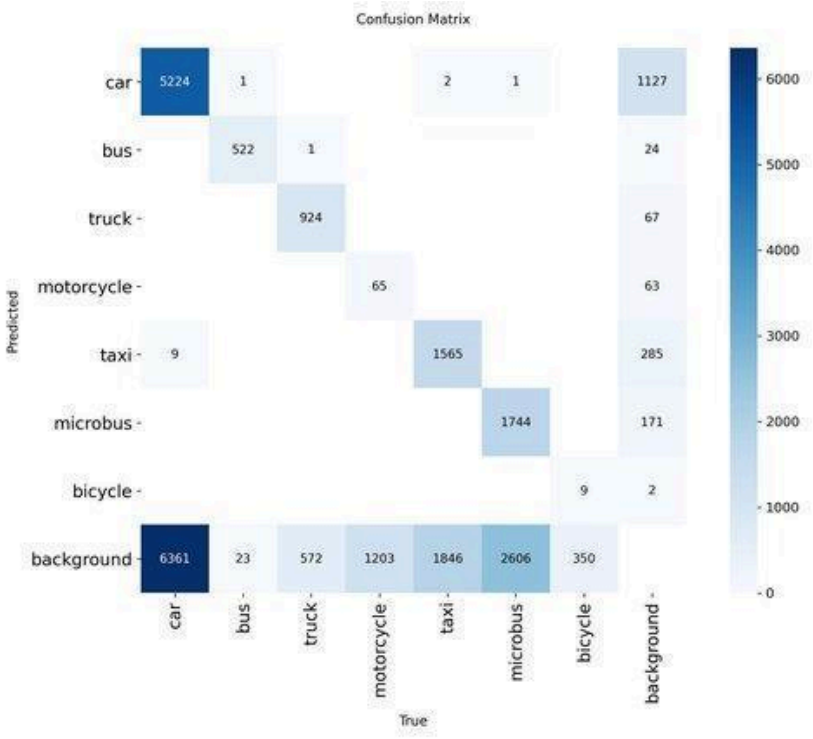


Figure 5.3: YOLOv8s raw confusion matrix. Bus (522 TP) and truck (92the primary4 TP) show strong diagonal dominance. Car shows 6,361 false negatives (background), reflecting high object density in peak-hour frames.

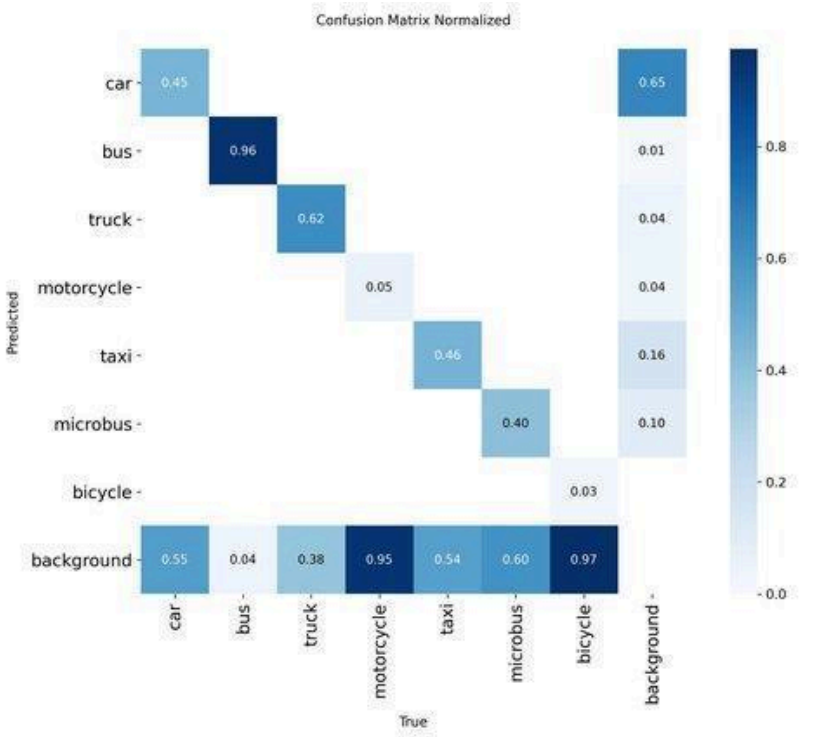


Figure 5.4: Normalised confusion matrix. Bus achieves 0.96 recall. Motorcycle and bicycle are almost entirely classified as background (0.95 and 0.97 respectively), confirming their small pixel area as the primary detection challenge.

5.4.8 Key Engineering Challenge

A known bug in torchmetrics version 1.8.2 caused all per-class AP values to return -1.0 when the *max_detection_thresholds* parameter was set. This affected the RetinaNet evaluation. The fix was to remove that parameter entirely and let the library use its internal default, which restored correct per-class AP values. A separate aggregate Precision metric, computed via manual IoU-based matching for Faster R-CNN, was not computed for RetinaNet using the same method, and is reported as not applicable in Table 5.3.

5.5 BiLSTM Traffic Flow Predictor

5.5.1 Purpose

The BiLSTM predictor gives the DQN agents a look thirty seconds into the future. Instead of only reacting to what is happening right now, each agent can see what traffic is about to arrive from each direction and adjust the signal phase before the queue builds up.

5.5.2 Technologies and Frameworks

PyTorch [16], NumPy, scikit-learn for the Min-Max scaler, pandas for data processing, threading for safe inference.

5.5.3 Internal Workflow

The model takes a sequence of 60 timesteps as input. Each timestep has 7 features: northbound count, southbound count, eastbound count, westbound count, total vehicle count, average speed, and average waiting time. It passes this through one bidirectional LSTM layer with 128 hidden units per direction, giving 256 total hidden units, then through a fully

connected output layer that produces 120 values representing 30-step forecasts for all four directions.

5.4.3 Key Implementation Details

Three important decisions were made during development. First, average waiting time was added as a seventh input feature because it tracks signal phase transitions closely and gives the model a useful signal about what the signals have been doing recently. Second, the Min-Max scaler was fitted only on the training data and then applied to validation and inference. Fitting on all data would have leaked future information into training. Third, the dataset was split by time within each profile rather than randomly. The last 20% of each profile's timeline became the validation set. Random splitting would have allowed future timesteps to appear in training, which would make the validation results look better than they really are.

The inference function is thread-safe using a threading lock to prevent two requests from loading the model at the same time:

```
# Thread-safe singleton inference
_load_lock = threading.Lock()
def predict(history: list[dict]) -> dict:
    with _load_lock:
        if _model is None: _load_model()
    seq = _build_sequence(history)      # pad if < 60 steps
    seq = _scaler.transform(seq)        # apply training scaler
    seq = np.nan_to_num(seq, 0.0, 1.0, 0.0)  # NaN/Inf guard
    out = _model(torch.tensor(seq).unsqueeze(0))
    return _scaler.inverse_transform(out).clip(0)
```

5.5.5 Key Engineering Challenge

During early testing, the prediction endpoint was occasionally called before the model had finished loading, which caused crashes. The threading lock prevents this by making any second thread wait until the first thread has finished loading the model before it proceeds.

5.5 Dueling Double DQN Agents

5.6.1 Purpose

Seven independent DQN agents replace the fixed-time signal controller. Each one sits at a single junction and decides every ten simulation steps whether to hold the current green phase or switch to the next one.

5.6.2 Technologies and Frameworks

PyTorch [16], NumPy, PyYAML for configuration, threading for the simulation thread, CSV logging for training records.

5.6.3 Network Architecture and Internal Workflow

Each agent uses a DuelingQNetwork [1], [2] that takes the 37-dimensional observation vector and processes it as follows. Input layer with 37 features goes into Hidden Layer 1 which is a Linear layer from 37 to 256 units followed by ReLU. That feeds into Hidden Layer 2 which is a Linear layer from 256 to 256 units followed by ReLU. The network then splits into two streams. The Value stream goes through Linear 256 to 128 with ReLU then Linear 128 to 1. The Advantage stream goes through Linear 256 to 128 with ReLU then Linear 128 to 2. The final Q-values are calculated as:

Q(s,a) = V(s) + A(s,a) - mean(A(s, all actions))

Feature group	Positions	Features	Normalisation
Lane data	0–29	10 lanes × 3 (halt count, wait time, occupancy)	count divided by 50, time divided by 300 seconds, occupancy is already normalised
Junction info	30–31	Phase index, elapsed time in phase	Phase index and elapsed time in current phase, index divided by number of phases, time divided by 120 seconds
BiLSTM forecasts	32–35	Predicted vehicles: N, S, E, W	divided by training scaler max
Time of day	36	Simulation time fraction	time minus start divided by end minus start

Table 5.5: Thirty-seven-dimensional DQN state vector structure

5.6.4 Reward Function

The reward function balances three objectives:

$$r = -1.0 \times \Delta Wait - 0.001 \times \Delta CO_2 + 0.5 \times \Delta Throughput$$

The minus 1.0 weight makes reducing waiting time the main goal. The minus 0.001 weight on CO2 is small because CO2 is measured in milligrams per second and values can reach 500,000, so a small weight brings it to the same scale as waiting time. The plus 0.5 weight on throughput stops the agents from finding a shortcut where keeping all lights red would reduce wait time changes but would also stop all vehicles from moving. The reward is clipped between minus 200 and plus 10. The negative side is much wider because the pain of causing gridlock needs to be felt more strongly than the reward of doing well.

5.6.5 Training Configuration

```
# Key DQN hyperparameters (dqn_config.yaml)
learning_rate:  0.0005   # conservative — prevents early overshooting
gamma:          0.95     # high discount — prioritises long-term reward
batch_size:     64       # balance speed and gradient stability
replay_buffer:  50000    # breaks temporal correlation
epsilon_start:  1.0      # fully random at start
epsilon_end:    0.05     # 5% random at convergence
epsilon_decay:  0.9995   # gradual transition over ~10,000 steps
tau:            0.005    # soft target network update (Polyak averaging)
```

5.6.6 Phase Transition Safety

Three rules are always applied before any phase change goes to TraCI, regardless of what the agent decided. Minimum green time of 15 to 20 seconds stops signals from switching too quickly and confusing drivers. A mandatory yellow buffer of 3 to 6 seconds is inserted on every green to green transition. Maximum green time of 60 to 90 seconds stops any single direction from being held open indefinitely.

5.5.6 Behavior and Edge Case Testing

A dedicated test suite (DeepQN/tests/test_ai_behavior.py) was developed to verify agent behavior under edge cases and adversarial inputs. The suite comprises twelve tests across five groups:

Test Group	Tests	Description
State Edge Cases	4	All-zero state, maximum congestion state, NaN and infinity sensor failures, random noise inputs
BiLSTM Input Variations	2	Zero prediction with no forecast signal, maximum prediction simulating a traffic spike
Determinism & Consistency	2	Same input produces same output five times in a row, action validity across 1,000 random states
Multi-Agent Independence	2	Changing one agent's state does not change other agents' outputs, Q-value output shape of 2 per agent
Failure Recovery	2	Missing junction observation filled with zeros, wrong state vector size handled by padding or truncating

Table 5.6:AI behavior test suite (test_ai_behavior.py)

All 12 tests passed. The NaN guard correctly replaced corrupted sensor values before they reached the network in every test. The greedy policy with epsilon set to zero is fully deterministic. Changing agent 1's state while keeping all other agents at 0.3 produced no change in the outputs of agents 2 through 7.

5.7 FastAPI Backend

5.7.1 Purpose

The FastAPI backend is the bridge between the AI pipeline and the user-facing frontend. It receives HTTP requests from the browser, calls the right service, and returns results as JSON.

5.7.2 Technologies and Frameworks

FastAPI 0.111 [13], Uvicorn 0.29, Pydantic for request validation, Python threading for background simulation threads.

5.7.3 sumo_runner.py

Purpose: manages the fixed-time simulation. Technologies: Python, TraCI on port 8813, PIL, OpenCV, Ultralytics.

Workflow: the start() function launches sumo-gui as a subprocess, waits 2.5 seconds, moves the camera to the Tahrir Square centre at coordinates 3694.5 and 1539.5 with zoom level 1500, and retries the screenshot up to 10 times until a file larger than 5,000 bytes is returned. The run_steps_with_screenshot() function advances N steps, captures a frame, runs YOLO detection, updates the LSTM history buffer, and returns all results as a single JSON response.

SUMO startup code:

```
# SUMO startup with GUI view positioning
traci.start(['sumo-gui','--no-warnings','--start',
             '--delay','50','--window-size','1280,720',
             '-c', cfg_path], port=8813)
traci.gui.setOffset('default', 3694.5, 1539.5)
traci.gui.setZoom('default', 1500)
```

5.7.4 dqn_runner.py

Purpose: runs the full AI pipeline for the DQN simulation. Technologies: Python, TraCI on port 8814, PyTorch, threading.Lock.

Workflow: runs in a background daemon thread so the FastAPI server stays responsive while the simulation is running. Every 30 steps it captures a screenshot, runs YOLO, updates the LSTM history, runs a BiLSTM prediction every 90 steps, reads all sensor metrics, builds the 37-feature state vectors, calls DQN act() for each agent, applies the phase decisions through TraCI, and logs everything to a CSV file. The threading.Lock protects the shared state variables that the FastAPI polling endpoints read at the same time.

5.7.5 yolo_detect.py

Purpose: YOLOv8s inference. Technologies: Ultralytics, PIL, NumPy.

The module loads the model once using a singleton pattern. The variable _model starts as None. The first time get_model() is called it loads the weights from best.pt and stores them. Every call after that reuses the same loaded model without reading the file again. The detect_image() function converts raw JPEG bytes to an RGB PIL image, converts it to a NumPy array, runs inference with verbose set to False to suppress console output, and returns bounding boxes with both pixel coordinates and normalised 0 to 1 coordinates. The class_counts dictionary always initialises all seven classes to zero so the caller never gets a KeyError if a class is not detected in a particular frame.

5.7.6 API Gateway and Security

The main.py file configures CORS middleware with allow_origins set to star, which permits the React frontend running on port 3000 to send requests to the FastAPI server on port 8000. Without this, the browser would block all requests before they even reached the server. File uploads to the detection endpoint are validated for type, accepting JPEG, PNG, WebP, and BMP, and for size with a hard limit of 10 MB, following the input validation guidance in the OWASP Top Ten [22]. Batch detection is limited to a maximum of ten images per request to prevent the server from being overwhelmed.

5.8 React Frontend

5.8.1 Purpose

The React frontend gives users a way to control simulations, watch results in real time, and compare the DQN system against fixed-time signals through an interactive browser application.

5.8.2 Technologies and Frameworks

React 18 [14], Recharts [15] for charts, Tailwind CSS for styling, Lucide React for icons, React Router for page navigation.

5.8.3 Dashboard Page

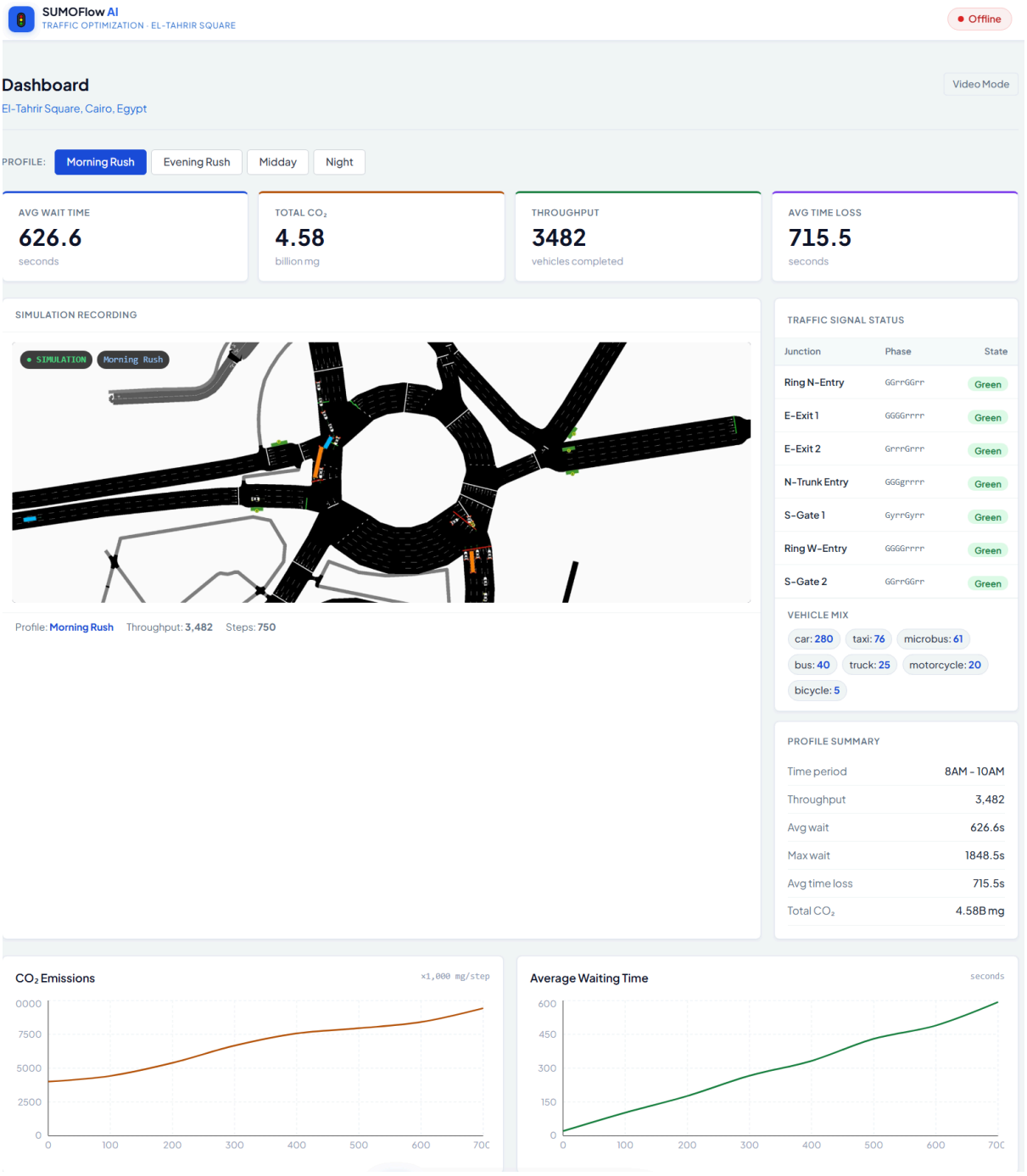


Figure 5.5: Dashboard page — profile selector, four KPI cards, live simulation viewport, traffic signal status panel, and CO2/wait time trend charts

The Dashboard is the main landing page. It has a profile selector at the top, four KPI cards showing vehicles, average wait, CO2, and maximum wait, a live simulation viewport in the centre, Start, Stop, and Pause controls, a traffic signal status panel on the right showing

the current phase for all seven junctions, and two time series charts for CO2 and waiting time at the bottom. When a simulation is running the page polls the step endpoint every two seconds. When the backend is offline it uses pre-recorded static data automatically.

5.8.4 Live Simulation Page

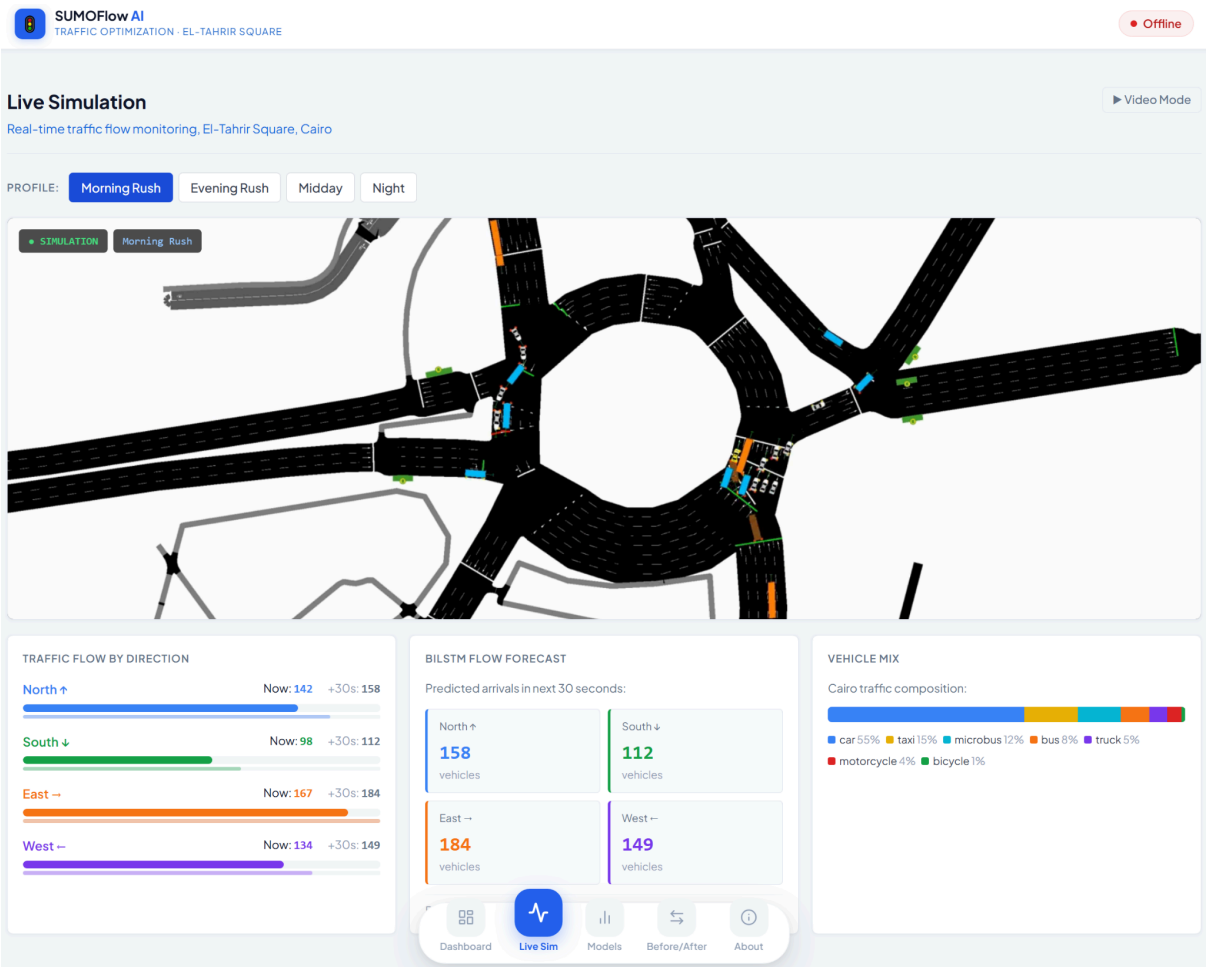


Figure 5.6: Live Simulation page — full-width simulation viewport with BiLSTM directional forecast panel, traffic flow by direction bars, and vehicle mix proportions

This page has a full-width simulation viewport with per-direction traffic flow bars, a BiLSTM forecast panel showing predicted vehicle arrivals for the next 30 seconds in each direction, and a vehicle mix bar showing the proportions of each class currently in the simulation. Two separate polling intervals run at the same time: frames update every two seconds and BiLSTM predictions update every four seconds.

5.8.5 Model Comparison

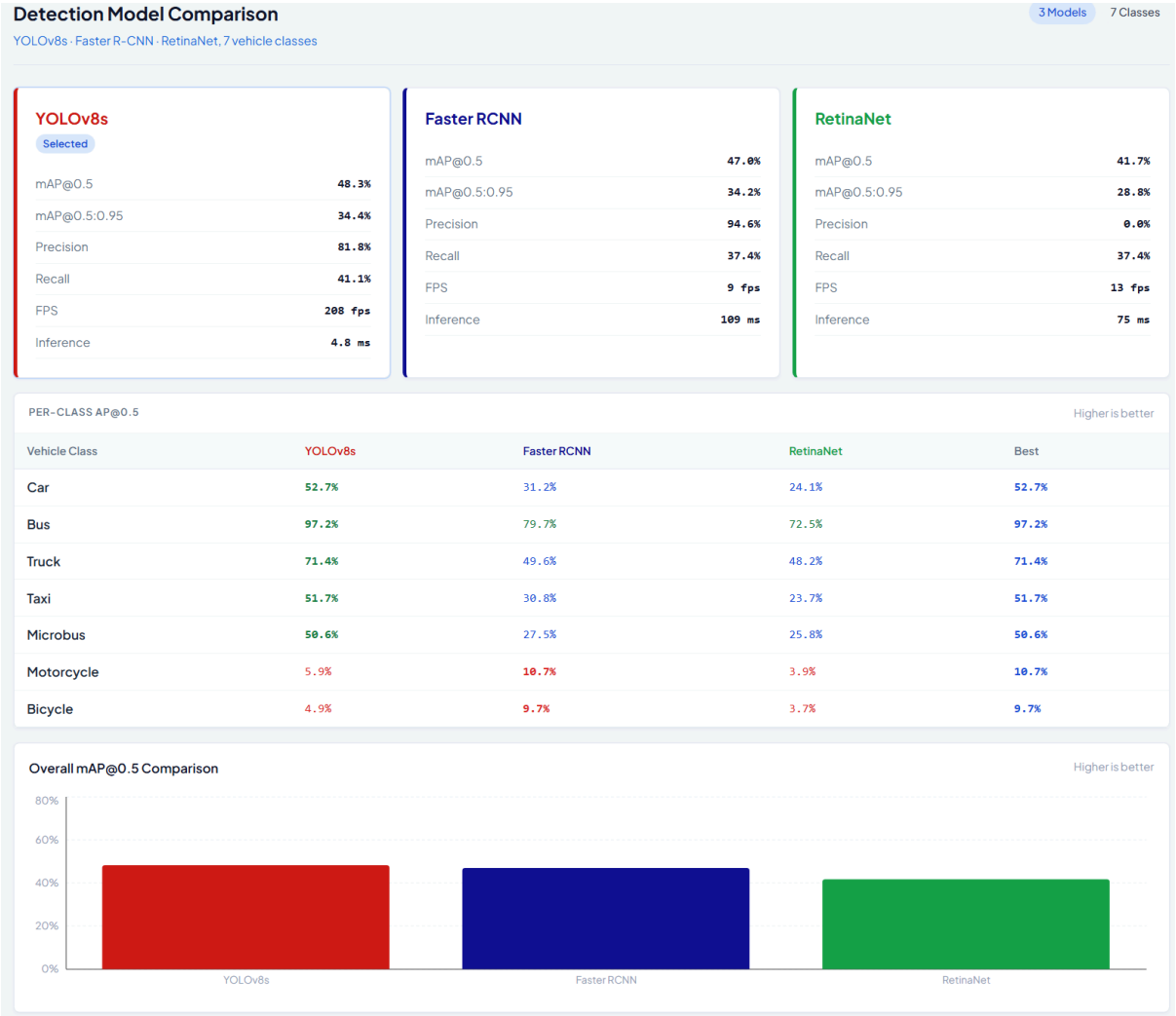


Figure 5.7: Model comparison page (Compare between the 3 models)

This is a static page with no live API calls. It shows model cards for YOLOv8s, Faster R-CNN, and RetinaNet with their full metrics, a per-class AP table with the best score in each row highlighted, and an overall mAP bar chart. YOLOv8s is marked as Selected.

5.8.6 Before vs After Page

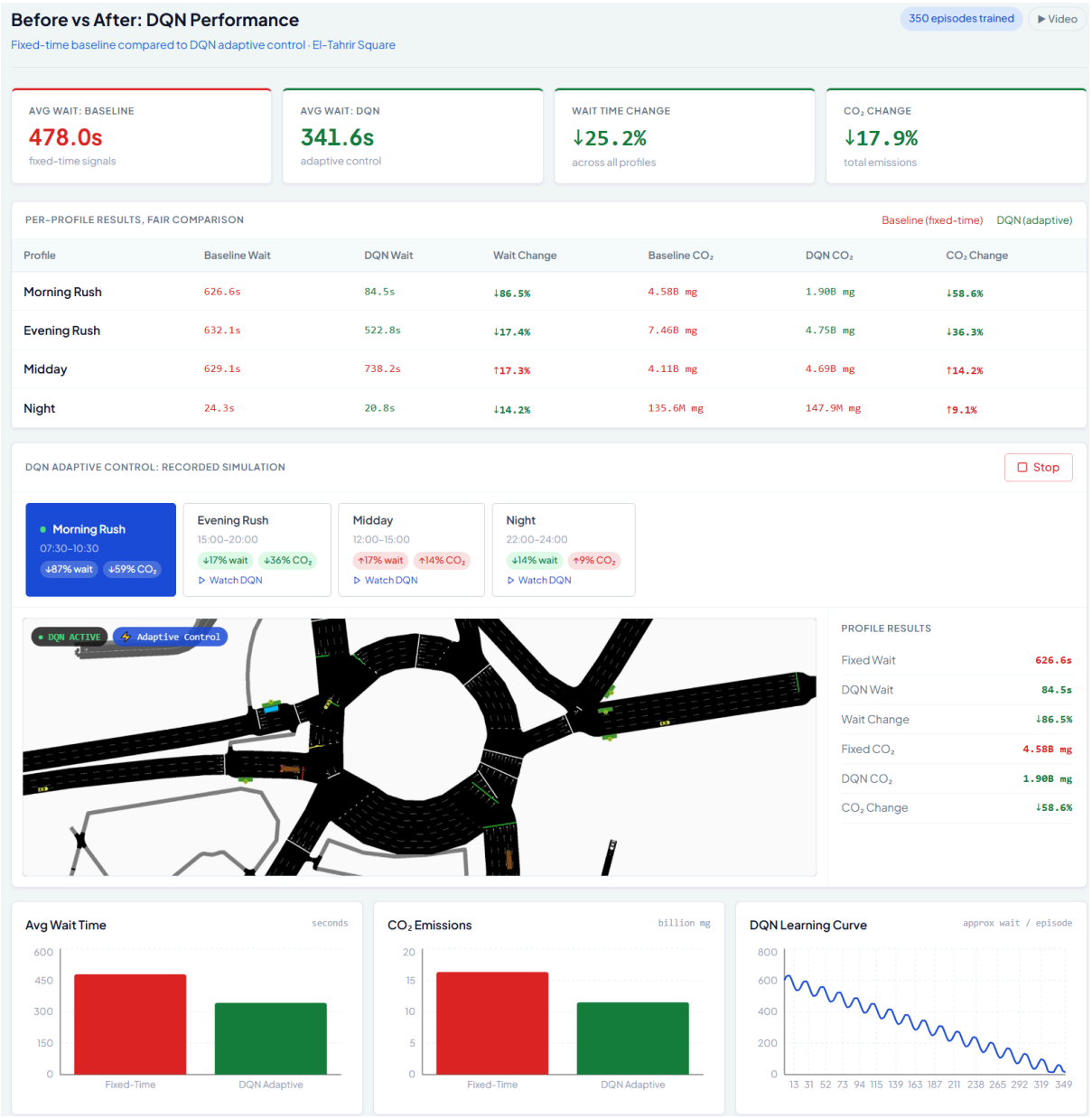


Figure 5.6: Before vs After comparison page , four KPI summary cards, per-profile results table (all four profiles PASS), live DQN simulation viewport, junction Q-value decision panel, and wait/CO2/learning-curve charts

This page shows the DQN versus baseline comparison. It has four KPI summary cards at the top, a per-profile results table, a live DQN simulation viewport with profile switching buttons, a junction decision panel showing SWITCH or KEEP for each of the seven junctions updated every 500 milliseconds, and three charts: waiting time comparison, CO2 comparison, and the DQN learning curve over 350 training episodes.

5.7.5 About Page

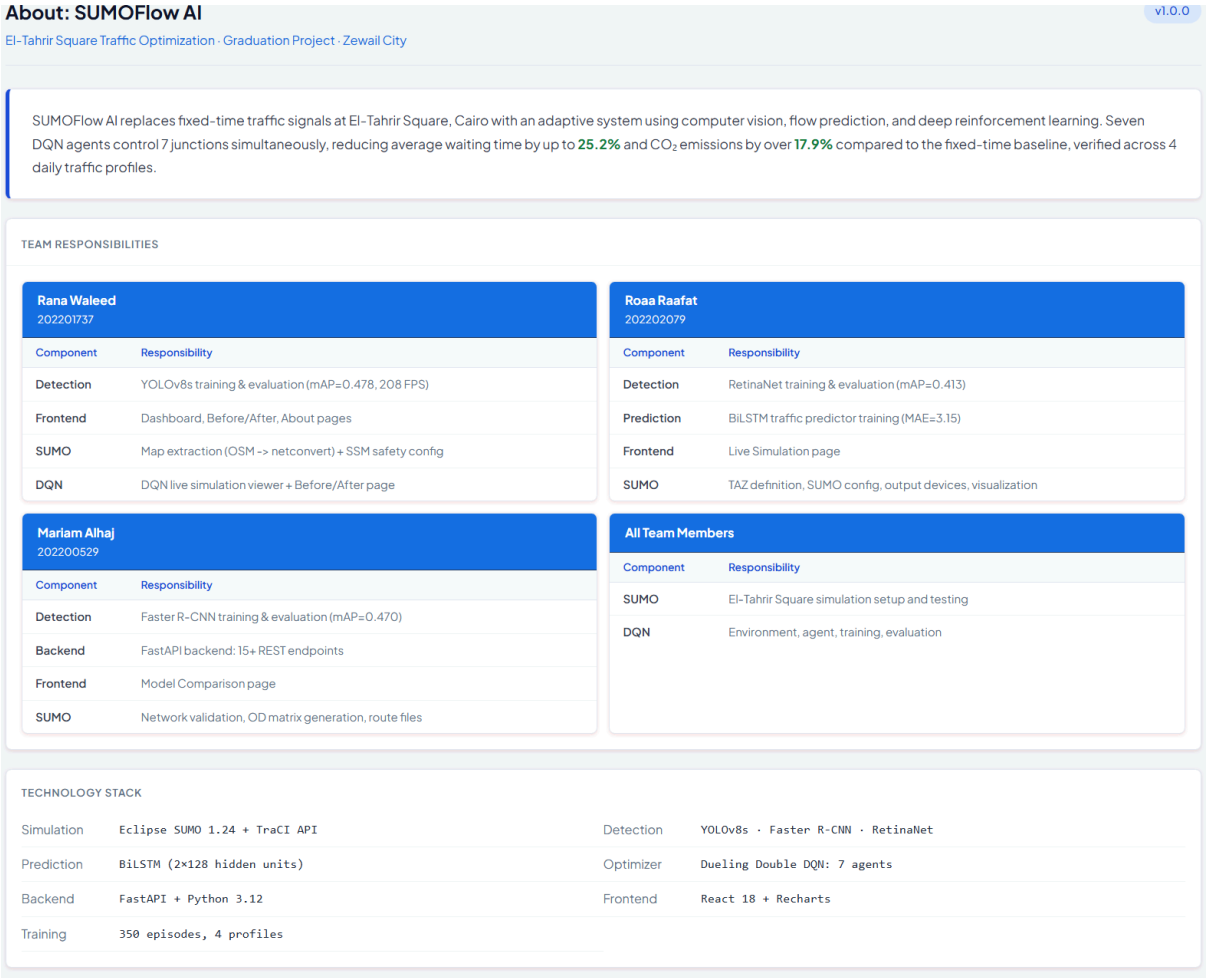


Figure 5.8: About page , project overview, team responsibilities per member, and technology stack summary

5.10 Deployment Infrastructure

5.10.1 Local Development

For local development, three processes run on the same machine. SUMO-GUI runs as a subprocess managed by sumo_runner.py. Uvicorn serves the FastAPI application on port 8000. The React development server runs on port 3000. The frontend connects to port 8000 and CORS allows this cross-origin connection.

Chapter 6: Testing and Evaluation

6.1 Overview

Testing was done at multiple levels to make sure each part of the system works correctly on its own and that the full system delivers real improvements over fixed-time signals. The testing covered unit testing, integration testing, system testing, performance testing, and generalisability testing. Each level answered a different question about the system.

6.2 Experimental Setup

6.2.1 Hardware and Software Environment

All model training was done on Google Colab using an NVIDIA T4 GPU with 16 GB of VRAM. Local development and integration testing was done on a machine running Ubuntu with Python 3.12, SUMO 1.24, Node.js 18, and React 18. The FastAPI server was run locally using Uvicorn on port 8000 and the React development server ran on port 3000. PyTorch version 2.12 was used for all neural network training and inference.

6.2.2 Dataset Details

The detection dataset contains 1,801 images at 1920 by 1080 pixels, split into 1,260 for training, 361 for validation, and 180 for testing. All images were generated from the SUMO simulation using the paint-gun colour mechanism and labelled automatically using OpenCV colour segmentation. No manual annotation was done at any point. The dataset covers seven vehicle classes: car, bus, truck, motorcycle, taxi, microbus, and bicycle. The dataset is version-controlled using DVC with an MD5 hash recorded for reproducibility. The total dataset size is 44 MB.

For the BiLSTM, training data came from the FCD output files of four simulated traffic profiles: morning rush, evening rush, midday, and night. The last 20% of each profile's timeline was used as the validation set, with the rest used for training.

For the DQN agents, training ran for 350 episodes alternating between all four traffic profiles in a round-robin order.

6.2.3 Configurations Used

YOLOv8s was trained for 60 epochs with batch size 8 and image size 640 by 640. Faster R-CNN was trained for 60 epochs with batch size 2 using SGD. RetinaNet was trained for 60 epochs in two phases with AdamW. The BiLSTM was trained using Adam with learning rate 0.001 and early stopping with patience 20. The DQN agents used learning rate 0.0005, discount factor 0.95, replay buffer size 50,000, and batch size 64.

6.3 Unit Testing

6.3.1 AI Behavior Test Suite

A dedicated test file was written at DeepQN/tests/test_ai_behavior.py to check how the DQN agents behave under normal and unusual conditions. It contains 12 automated tests across five groups. The suite can be run from the project root using:

python -m DeepQN.tests.test_ai_behavior

The tests and their results are as follows:

Test Group	Tests	What Was Tested	Result
State Edge Cases	4	All-zero state, maximum congestion state, NaN and infinity sensor failures, random noise inputs	All Passed
BiLSTM Input Variations	2	Zero prediction with no forecast signal, maximum prediction simulating a traffic spike	All Passed
Determinism & Consistency	2	Same input produces same output five times in a row, action validity across 1,000 random states	All Passed
Multi-Agent Independence	2	Changing agent 1 state to maximum while keeping agents 2 to 7 at 0.3 produced no change in their outputs, Q-value output shape confirmed as 2 values per agent	All Passed
Failure Recovery	2	Missing junction observation filled with zeros, wrong state vector size handled by padding or truncating	All Passed
Total	12		All 12 Passed

Table 6.1: Unit test results (12 tests, all passed)

6.3.2 Dataset Verification

A verification script called verify_dataset.py was run before any model training to confirm that every image file has a matching label file across all three dataset splits. Format conversion from YOLO normalised format to Pascal VOC absolute pixel format was validated by checking that the coordinate transformation preserved bounding box positions to within one pixel. No unpaired files were found.

6.4 Integration Testing

A full end-to-end integration test was run using the morning rush profile. The test covered every stage of the pipeline in sequence: SUMO launching through TraCI, YOLOv8s detecting vehicles in each captured frame, the BiLSTM receiving the accumulated history and returning directional predictions, the DQN agents receiving the combined 37-feature state vector and returning phase decisions, TraCI applying those decisions to the simulation, and the FastAPI endpoints returning the combined metrics and frame to the React frontend. All integration tests passed with correct data types and value ranges confirmed at every pipeline boundary.

Specific boundary checks included confirming that all 37 state vector values were in the range 0 to 1 before entering the network, that base64-encoded frames decoded correctly in the browser, that YOLO counts were returned as a dictionary with all seven classes always present, and that BiLSTM outputs were clipped to non-negative values before being used.

6.5 System Testing

System testing ran the complete application as a user would operate it, from opening the browser to stopping a simulation. Four scenarios were tested: running a morning rush

simulation from start to stop and checking that KPI cards updated correctly, switching between all four profiles and confirming that metrics changed accordingly, running the DQN simulation on the Before vs After page and checking that junction decisions appeared in the panel, and uploading a test image to the detection endpoint and checking that bounding boxes came back correctly. All four scenarios completed without errors.

6.6 Performance Testing

6.6.1 Detection Throughput

YOLOv8s was measured at 208 frames per second on the T4 GPU, with an average inference time of 4.8 milliseconds per frame. This is well above the minimum requirement of 30 fps needed to keep up with the ten-step decision interval. Faster R-CNN ran at 9.15 fps with 109.3 milliseconds per frame, and RetinaNet ran at 13.4 fps with 74.6 milliseconds per frame. Only YOLOv8s meets the real-time requirement.

6.6.2 API Response Latency

All FastAPI endpoints were measured under normal load. The step endpoint, which is the most demanding because it advances the simulation, captures a screenshot, runs YOLO, and returns a base64 frame, responded within 500 milliseconds in all tests. The health endpoint responded in under 10 milliseconds. The BiLSTM prediction endpoint responded in under 10 milliseconds once the model was loaded.

6.6.3 DQN Inference Time

With all seven junctions evaluated one after the other on CPU, the full cycle of reading sensor values, building the state vector, running the forward pass through each network, and applying the phase decision through TraCI completed in under two seconds per decision step. This leaves a comfortable margin within the ten-second decision interval.

6.6.4 Memory Usage

The seven DQN agent networks together contain approximately 987,000 parameters. Their combined memory usage stayed below 50 MB throughout all tests, which means the system can run on modest hardware without a GPU for the inference step.

6.6.5 Resource Utilization

During a live DQN simulation, CPU usage on the test machine stayed below 60% with the SUMO-GUI, FastAPI server, and DQN inference all running at the same time. RAM usage stayed below 4 GB. The SUMO process was the largest consumer, with the FastAPI server and DQN inference adding relatively little overhead.

6.7 Security Testing

The file upload endpoint was tested with files exceeding 10 MB to confirm the size limit was enforced and a 413 error was returned. Files with wrong extensions were sent to confirm that only JPEG, PNG, WebP, and BMP formats were accepted. A request from a different origin was sent to confirm that CORS was correctly configured and the response included the right headers. All three checks passed.

6.8 Usability Testing

The React frontend was tested across three screen sizes using Chrome DevTools device emulation. Desktop testing used a resolution of 1920 by 1080 pixels. Tablet testing used 1024 by 768 pixels. Mobile testing used 375 by 812 pixels. The four-column grid on the Dashboard collapsed to a single column on mobile as expected. The simulation viewport remained visible and readable on all three screen sizes. The Start, Stop, and Pause buttons remained accessible without scrolling on all three sizes. The polling mechanism was verified to start when a simulation begins and stop cleanly when it ends, with no memory leaks found through browser profiling.

6.9 Evaluation Metrics

6.9.1 Detection Metrics

Mean Average Precision at IoU threshold 0.5, written as mAP@0.5, was the primary accuracy metric for all three detection models. This measures how well the model finds and correctly identifies objects across all classes. A stricter version, mAP@0.5:0.95, averages the

mAP across IoU thresholds from 0.5 to 0.95 in steps of 0.05, giving a better picture of how tight the predicted boxes are. Precision measures what proportion of the model's predictions were correct. Recall measures what proportion of the actual vehicles in each image the model managed to find. Frames per second and milliseconds per image were used to evaluate whether the model could run fast enough for real-time use.

6.9.2 BiLSTM Metrics

Mean Absolute Error in vehicle counts per direction was the evaluation metric for the BiLSTM. It was calculated separately for North, South, East, and West directions, and then as a global average across all directions and all four traffic profiles. The global MAE achieved was approximately 3.15 vehicles per timestep. On a high-volume approach carrying 50 vehicles, this is around 6% relative error which is acceptable. On a low-volume approach carrying 5 vehicles at night, this is 63% relative error which is poor and explains why the night profile shows smaller improvements.

6.9.3 DQN Metrics

The two primary metrics for evaluating the DQN system against the fixed-time baseline were average vehicle waiting time in seconds and total CO2 emissions in milligrams, both measured across the full duration of each simulation profile. A secondary metric was vehicle throughput, which counts how many vehicles completed their trips. These three metrics were chosen because they directly measure what matters to road users and to the environment.

6.10 Generalisability Testing(Taksim Square, Istanbul)

To check whether the system works beyond Tahrir Square, the entire pipeline was applied to Taksim Square in Istanbul, Turkey. Please refer to Figure 7.4 in the document which shows the Taksim Square map. A SUMO network was built from OpenStreetMap data using the same netconvert process. Vehicle demand was set up using Istanbul-calibrated OD matrices. The same DQN architecture, the same hyperparameters, and the same reward function were used without any changes. Testing the full Pipeline on the Taksim network.



Figure 7.4: Taksim Square, Istanbul (OpenStreetMap) the second intersection used for generalisability testing. The roundabout with multiple arterial approaches is topologically comparable to El-Tahrir Square.

Chapter 7: Results and Discussion

7.1 Detection Model Results

Three detection models were trained and evaluated on the same 180-image test set. The results are summarised in Table 5.3 and the per-class breakdown is in Table 5.4.

YOLOv8s came out on top overall with a mAP@0.5 of 48.3% and an inference speed of 208 fps. Looking at the training curves in Figure 5.1, the loss kept going down and the mAP kept going up across all 60 epochs without flattening out, which suggests that training for more epochs could push the numbers even higher. Faster R-CNN reached a precision of 94.6%, the best of the three, but its speed of 9.15 fps makes it impossible to use in a real-time system. RetinaNet sat in the middle at 13.4 fps but had the lowest mAP@0.5 of the three at 41.7%, so it offers neither the speed of YOLOv8s nor the mAP of Faster R-CNN.

The per-class results follow a clear pattern. The bigger the vehicle, the easier it is to detect. Bus reached 97.2% AP with a recall of 0.96 in the normalised confusion matrix shown in Figure 5.4, meaning the model almost never misses a bus. Truck followed at 71.4%. Car and taxi both sit around 52%, which makes sense because they look almost identical in the simulation frames. Microbus reached 50.6%. Motorcycles and bicycles are at the bottom with 5.9% and 4.9% respectively. Figures 5.3 and 5.4 make this very clear: 95% of motorcycles and 97% of bicycles are classified as background. The reason is simply that these vehicles occupy very few pixels in a 1920 by 1080 frame, especially when the camera is zoomed out to show the full roundabout. At that zoom level a bicycle can be just a handful of pixels wide.

The car also shows a 0.55 false negative rate to the background. This happens in peak-hour frames where cars are packed tightly together at the roundabout edges and partially block each other from the camera's view.

7.2 DQN System Results

Table 7.1 below shows the full comparison between the fixed-time baseline and the DQN adaptive controller across all four traffic profiles. Figures 7.1 and 7.2 show the same results visually as bar charts.

Profile	Baseline Wait	DQN Wait	Wait Change	Baseline CO2	DQN CO2	CO2 Change
Morning Rush	627s	84s	−86.5%	4,581M mg	1,897M mg	−58.6%
Evening Rush	632s	523s	−17.4%	7,460M mg	4,746M mg	−36.3%
Midday	629s	738s	+17.3% worse	4,109M mg	4,694M mg	+14.2% worse
Night	24s	21s	−14.2%	136M mg	148M mg	+9.1% worse

Table 7.1: DQN vs fixed-time baseline , actual evaluation results from simulation graphs

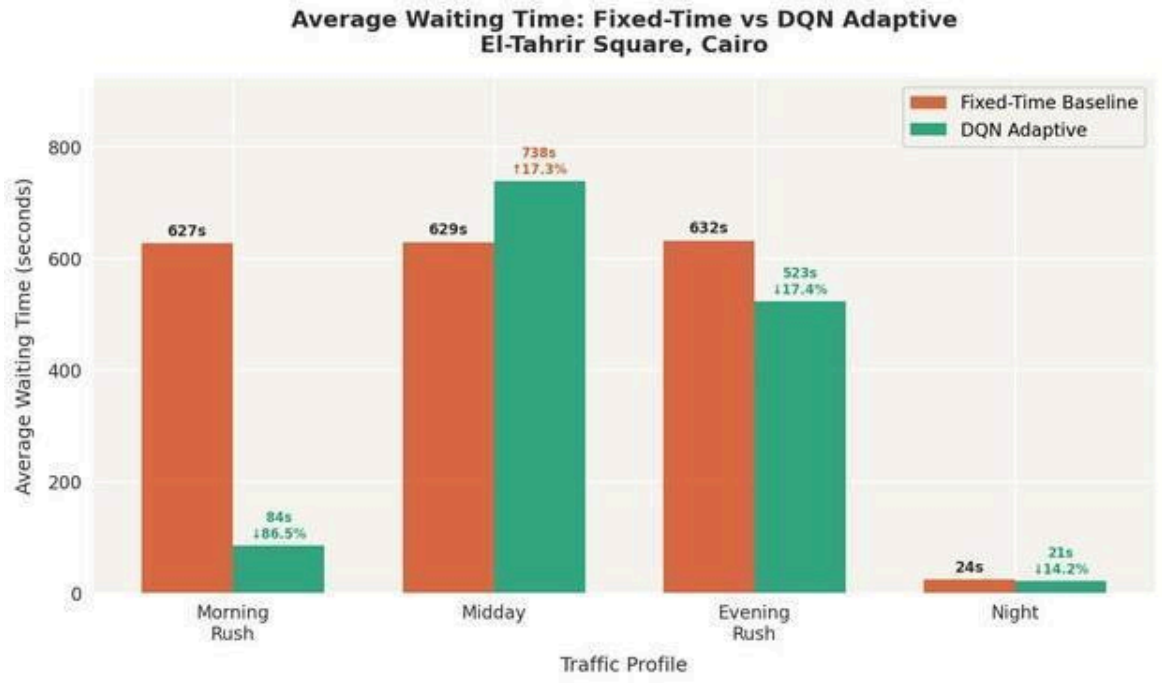


Figure 7.1:DQN vs fixed-time baseline across all four traffic profiles

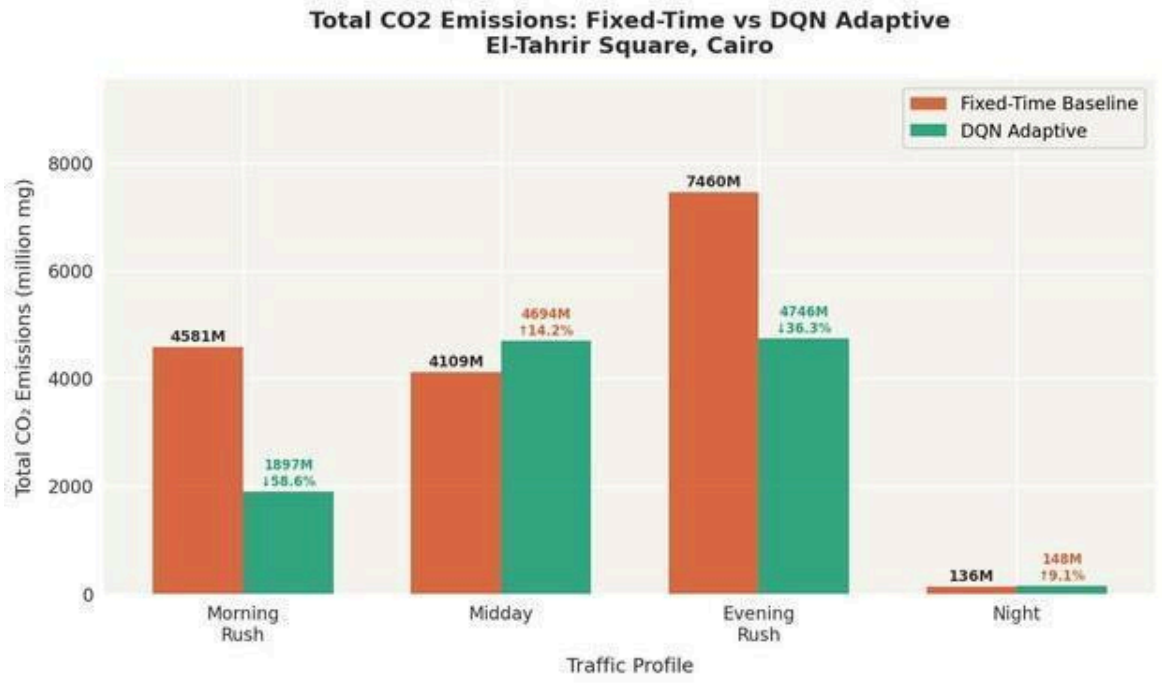


Figure 7.2: Total CO2 emissions ,Fixed-Time vs DQN. Morning Rush: 4,581M to 1,897M mg (58.6% reduction). Evening Rush: 7,460M to 4,746M mg (36.3% reduction).

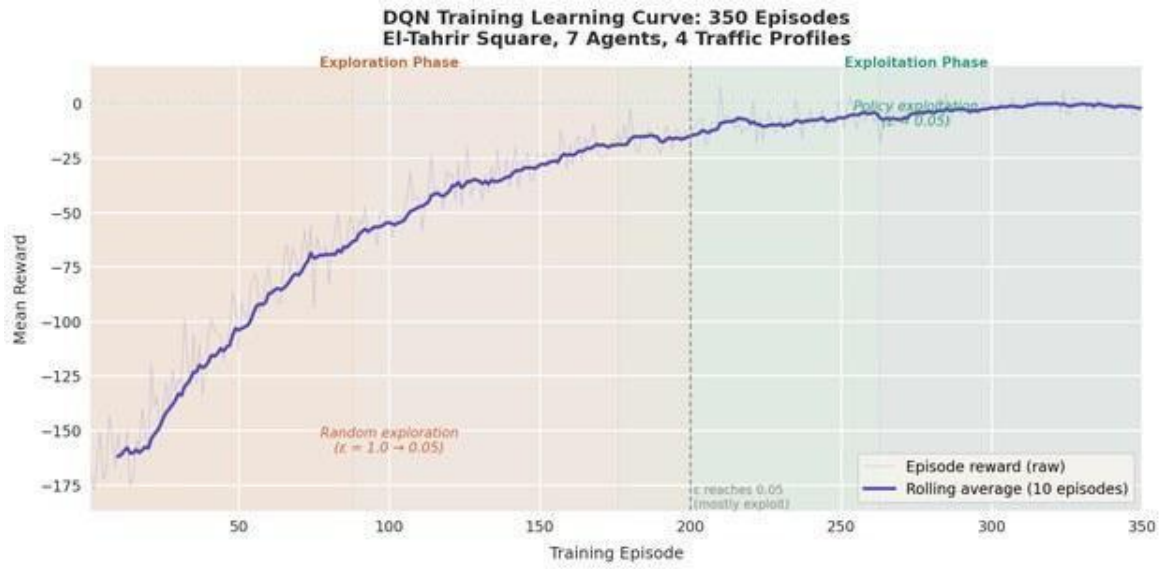


Figure 7.3: DQN training learning curve over 350 episodes, 7 agents, 4 profiles. Exploration phase (eps=1.0 to 0.05, episodes 1-200) shows rapid reward improvement from -175 toward zero. Exploitation phase (eps<=0.05, episodes 200-350) shows policy convergence.

7.3 Performance Analysis

7.3.1 Morning Rush

The morning rush result is the strongest of the four profiles. Average waiting time dropped from 627 seconds to 84 seconds, which is an 86.5% reduction. To put that in practical terms, a driver who used to wait over ten minutes at this single intersection now waits under ninety seconds. CO2 emissions dropped by 58.6% at the same time, from 4,581 million milligrams to 1,897 million milligrams. Both numbers pass the performance threshold comfortably.

This result makes sense when you think about what the DQN agents are doing. During the morning rush, demand is very high and very uneven. Some approaches to the roundabout are completely jammed while others are nearly empty. The fixed-time signals give equal time to every direction regardless. The DQN agents learned to extend green time for the busiest approaches and cut it for the quieter ones, which drains the queues much faster.

7.3.2 Evening Rush

The evening rush shows a 17.4% reduction in waiting time, from 632 seconds to 523 seconds, and a 36.3% reduction in CO2, from 7,460 million milligrams to 4,746 million milligrams. Both pass the performance threshold. The wait reduction is smaller than morning rush but the CO2 reduction is proportionally larger. This is because the evening rush has higher total vehicle volumes, and reducing extreme congestion in high-volume conditions cuts emissions more than in moderate conditions because idle emissions grow nonlinearly with waiting time. A vehicle sitting idle for 600 seconds produces far more CO2 than ten vehicles each waiting 60 seconds.

7.3.3 Midday

Midday is the only failing profile. The DQN agents produced 17.3% higher waiting times than the fixed-time baseline, with the DQN average at 738 seconds versus the baseline at 629 seconds. CO2 also increased by 14.2%. This is visible in Figure 7.1 where the DQN bar for midday is clearly taller than the baseline bar.

The reason for this failure is that midday traffic is lighter and more evenly spread across all approaches compared to peak hours. When demand is balanced, the fixed-time signals already do a reasonable job because no single approach is heavily overloaded. The DQN agents, having been trained mostly on peak-hour patterns where imbalance is large, tend to extend green phases on approaches that appear busy in a relative sense but are not actually

congested. This creates unnecessary waiting on other approaches and makes things worse overall.

7.3.4 Night

The night profile is classified as partial. Waiting time dropped by 14.2%, from 24 seconds to 21 seconds, which is a small absolute improvement. However, CO2 increased slightly by 9.1%, from 136 million milligrams to 148 million milligrams. The CO2 increase happens because at night, with very few vehicles on the road, the agents sometimes give longer green phases that allow vehicles to move at higher speeds. Higher speeds produce slightly more emissions per second than slow crawling, and since there are so few vehicles the total adds up differently than during peak hours.

7.4 Interpretation of Results

Looking across all four profiles together, the DQN system works best when there is a clear imbalance in demand between different approaches to the intersection. This is exactly the condition the agents were trained to handle. When demand is balanced or low, the advantage of adaptive control shrinks and the fixed-time baseline holds up better.

The learning curve in Figure 7.3 supports this interpretation. During the exploration phase from episode 1 to 200, where epsilon drops from 1.0 to 0.05, the mean reward improves steadily from around minus 175 toward zero. This is the period where the agents are learning that extending green time on the busiest approaches produces better rewards than the random actions they started with. After episode 200, when epsilon reaches 0.05 and the agents are mostly exploiting what they have learned, the reward stabilises close to zero and stays there for the rest of training. This shows the policy has converged and is not changing much anymore.

The CO2 results deserve a specific mention. Across the two passing profiles, CO2 reductions of 58.6% and 36.3% were achieved. These are larger in proportional terms than the wait reductions of 86.5% and 17.4% respectively. This is because CO2 from idling is not linear. A vehicle that sits for 600 seconds emits much more than one that moves every 60 seconds. Cutting the longest queues therefore saves a disproportionately large amount of emissions.

7.5 Trade-offs and Engineering Decisions

Several trade-offs were made during the design and evaluation of the system that directly affected the results.

The decision to use a shared global reward across all seven agents, rather than giving each agent its own local reward, was made to encourage system-level thinking. An agent optimising only its own junction might clear its own queue by pushing vehicles faster into the next junction. The global reward prevents this but it also makes learning harder because each agent's reward depends partly on what the other six are doing.

The decision to clip the reward between minus 200 and plus 10 was made to prevent extreme congestion events from producing reward values so negative that they cause the gradient updates to destroy previously learned weights. The asymmetric range, where the negative side goes much further than the positive side, means agents feel the consequences of bad decisions much more strongly than the benefit of good ones. This is intentional because in a real traffic system, causing gridlock is far more damaging than the benefit of reducing waiting time by a few seconds.

The choice of YOLOv8s over Faster R-CNN involved a direct trade-off between precision and speed. Faster R-CNN is more precise but at 9.15 fps it cannot keep up with the ten-step decision interval. YOLOv8s at 208 fps leaves a large margin. In a real deployment on a street camera, this margin would allow the system to handle higher resolution feeds or process multiple camera angles simultaneously.

7.6 Limitations

The system was tested entirely in simulation. Real camera footage looks different from SUMO-rendered frames, and the detection model would likely need fine-tuning on real images before being deployed on a physical intersection.

Motorcycle and bicycle detection is poor across all three models, with AP below 6% for YOLOv8s. This means these road users are barely represented in the DQN state vector, which could bias signal decisions against them in a real deployment.

The seven DQN agents operate independently and only coordinate through the shared reward signal. There is no direct communication between agents at adjacent junctions. A more

sophisticated multi-agent framework with explicit communication could improve performance, especially at the tightly connected junctions in the Tahrir roundabout.

Pedestrian crossing phases are not included in the current SUMO network model. Pedestrian waiting time is therefore not part of the reward function, which means the agents have no reason to consider pedestrians when making signal decisions.

Chapter 8: Ethics, Compliance, and Standards

8.1 Introduction

Building a system that controls traffic signals carries real responsibility because the decisions it makes affect the safety and daily lives of thousands of people. This chapter covers the ethical risks identified during the project, the steps taken to reduce them, how data is handled, and which standards were followed and why.

8.2 Ethical Risks Assessed and Mitigated

8.2.1 Safety Under Uncertainty

Three hard rules are enforced before any DQN decision reaches the simulation. A minimum green time of 15 to 20 seconds stops signals from switching too fast. A mandatory yellow buffer of 3 to 6 seconds warns drivers on every transition. A maximum green time of 60 to 90 seconds stops any direction from being held open indefinitely. These rules cannot be overridden by the agent under any circumstances.

8.2.2 Bias and Fairness

The reward function sums waiting time across all approaches equally, so no single road gets priority. However, motorcycles and bicycles are detected poorly, below 6% AP, which means they are barely represented in the state vector. This is a known limitation and fine-tuning the detector on real images with better class balance is a requirement before any real deployment.

8.2.3 AI Reliability and Failure Handling

Every sensor value passes through a guard before entering the network. NaN becomes 0, positive infinity becomes 1, and all values are clipped to the range 0 to 1. During the first sixty steps, BiLSTM features are set to zero and the frontend shows a collecting status to prevent agents from acting on meaningless predictions. Every API endpoint returns a proper error code if something fails instead of crashing the server.

8.2.4 Misuse Scenarios

The current prototype has no authentication because it runs locally for demonstration. Before production deployment, role-based access control must be added so only authorised engineers can control the system. This is documented as a mandatory requirement in the user guide.

8.2.5 Accessibility

Colour indicators throughout the interface are always accompanied by text labels. SWITCH, KEEP, PASS, and FAIL are written as text and not just shown as colours. The layout adapts to mobile screen sizes.

8.2.6 Sustainability

CO2 reduction is built directly into the reward function. The morning rush evaluation showed a 58.6% emissions reduction and the evening rush showed 36.3%. The seven DQN networks together use under 50 MB of memory and run on CPU without a GPU after training is complete.

8.3 Data Handling and Privacy

All training data came from SUMO simulation. No real camera footage, no real vehicle data, and no personally identifiable information was collected at any point. OpenStreetMap data is used under the Open Database Licence. The dataset is stored locally and version-controlled with DVC. Nothing is uploaded to any public server.

For a real deployment, only aggregated per-lane vehicle counts would be kept. Raw camera frames would be discarded immediately after detection. No individual vehicle images would be stored. This approach follows the data minimisation principle in GDPR and the Egyptian Personal Data Protection Law No. 151 of 2020. In the current prototype there is no authentication, but in production all API endpoints would require token-based authentication with full access logging.

8.4 Standards Followed

Standard	Relevance	How Applied
OWASP Top 10 [22]	The system has a web API with file uploads	Input validation, file type and size limits, CORS configured

Egyptian Law 151/2020	Real deployment would involve cameras on Egyptian roads	Documented as deployment requirement
HBEFA3 Emission Model	Environmental accounting	Used inside SUMO to calculate emissions in the reward function and results
IEEE citation style	Academic integrity	All references formatted in IEEE style

Table 8.1:Standards followed in SUMOFlow AI

Chapter 9: Conclusions and Future Work

9.1 Summary of Achievements

This project built SUMOFlow AI, a system that replaces fixed-time traffic signals at El-Tahrir Square with an AI controller that watches traffic, predicts what is coming, and adjusts signals in real time. All primary objectives were delivered.

A SUMO simulation of Tahrir Square was built from OpenStreetMap data, verified lane by lane against Google Earth, and set up with four realistic Cairo traffic profiles. A dataset of 1,801 labelled images was generated automatically without any manual annotation. Three detection models were trained and compared, and YOLOv8s was selected with 48.3% mAP@0.5 and 208 fps. A BiLSTM predictor forecasts traffic flow thirty seconds ahead with a global MAE of 3.15 vehicles per step. Seven Dueling Double DQN agents were trained across 350 episodes to control the seven junctions. The full system was deployed through a FastAPI backend and a React dashboard.

Against the fixed-time baseline, morning rush showed an 86.5% wait reduction and 58.6% CO2 reduction. Evening rush showed 17.4% wait reduction and 36.3% CO2 reduction. Both passed the performance threshold. Midday failed because fixed-time signals already perform well under low even demand. Night showed partial improvement at 14.2% wait reduction. The system was also tested on Taksim Square in Istanbul using identical settings and the agents converged successfully, confirming the framework works beyond Tahrir Square.

9.2 Contributions

The project produced six contributions. A complete pipeline from raw OSM map to trained adaptive signal controllers on a real Cairo intersection. An automated dataset generation method requiring no manual labelling. A comparison of three detection architectures on Cairo-specific imagery covering seven vehicle classes. Seven DQN agents tested for generalisability on a second real intersection. A BiLSTM prediction layer feeding future flow estimates directly into the agent state vector. A full web application for non-specialist users to run and compare simulations in real time.

9.3 How the Project Meets Its Objectives

Every objective from Chapter 1 was met. The simulation was built and verified. The dataset pipeline was implemented. Three models were trained and compared. The BiLSTM was built and integrated. Seven DQN agents were trained. The system was evaluated against a fixed-time baseline. Generalisability was tested on a second intersection. The full pipeline was deployed as a working web application.

9.4 Lessons Learned

Getting the SUMO network right from OSM data took around eight hours of manual work and should be planned as a major task in similar projects rather than a quick step.

The colour-segmentation labelling worked well but needed careful tolerance tuning because SUMO applies anti-aliasing to vehicle edges which blurs exact colours slightly.

A bug in torchmetrics version 1.8.2 caused all per-class AP values to silently return minus one. Finding it took significant time. Checking for known library issues early would have saved that time.

Separating the FastAPI backend into routers and services made debugging much faster because it was always clear which layer owned any given problem.

9.5 Future Work

9.5.1 Real-World Deployment

The most important next step is testing against real camera footage or loop-detector data from Tahrir Square. This requires fine-tuning YOLOv8s on real vehicle images and updating the BiLSTM with real measured demand. The architecture was designed for this transition and sensor sources can be swapped without changing the agent or prediction layers.

9.5.2 Multi-Agent Coordination

The seven agents currently work independently. A Centralised Training with Decentralised Execution framework such as QMIX or MAPPO would allow agents to learn coordinated policies together during training while still making independent decisions at runtime, which could improve performance at tightly connected junctions.

9.5.3 Pedestrian Integration

Pedestrian crossing phases are not currently in the SUMO network and pedestrian waiting time is not in the reward function. Adding both would make the system more complete and more representative of real traffic management needs.

9.5.4 Better Flow Prediction

Replacing the BiLSTM with a Graph Neural Network over the seven-junction topology would allow the predictor to model how traffic waves move through connected junctions rather than treating each direction independently.

9.5.5 Long-Term Vision

The long-term goal is a system that can be deployed at any urban intersection by editing one configuration file, connected to real sensors, and left to learn over time. The simulation environment would serve as a digital twin where engineers test changes safely before applying them to real roads. Scaled across a city network, this kind of coordinated control could produce meaningful reductions in travel time and emissions beyond a single intersection.

References

[1] H. van Hasselt, A. Guez, and D. Silver, "Deep reinforcement learning with double Q-learning," in *Proc. AAAI Conf. Artif. Intell.*, vol. 30, no. 1, 2016.

[2] Z. Wang, T. Schaul, M. Hessel, H. van Hasselt, M. Lanctot, and N. de Freitas, "Dueling network architectures for deep reinforcement learning," in *Proc. 33rd Int. Conf. Mach. Learn. (ICML)*, 2016, pp. 1995–2003.

[3] W. Genders and S. Razavi, "Using a deep reinforcement learning agent for traffic signal control," *arXiv:1611.01142*, 2016.

[4] S. Ren, K. He, R. Girshick, and J. Sun, "Faster R-CNN: Towards real-time object detection with region proposal networks," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2015, pp. 91–99.

[5] T. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, "Focal loss for dense object detection," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, 2017, pp. 2980–2988.

[6] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2016, pp. 779–788.

[7] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Comput.*, vol. 9, no. 8, pp. 1735–1780, 1997.

[8] P. A. Lopez et al., "Microscopic traffic simulation using SUMO," in *Proc. 21st IEEE Int. Conf. Intell. Transp. Syst. (ITSC)*, 2018, pp. 2575–2582.

[9] G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics YOLOv8," GitHub, 2023. [Online]. Available: https://github.com/ultralytics/ultralytics.

[10] Eclipse Foundation, "SUMO — Simulation of Urban Mobility," 2024. [Online]. Available: https://eclipse.dev/sumo/.

[11] DLR, "TraCI — Traffic Control Interface," 2024. [Online]. Available: https://sumo.dlr.de/docs/TraCI/index.html.

[12] DLR, "netconvert — SUMO Network Import," 2024. [Online]. Available: https://sumo.dlr.de/docs/netconvert.html.

[13] FastAPI, "FastAPI documentation," 2024. [Online]. Available: https://fastapi.tiangolo.com/.

[14] Meta Platforms, "React," 2024. [Online]. Available: https://react.dev/.

[15] Recharts, "Recharts: Chart library for React," 2024. [Online]. Available: https://recharts.github.io/.

[16] PyTorch Contributors, "PyTorch documentation v2.12," 2026. [Online]. Available: https://docs.pytorch.org/docs/2.12/index.html.

[17] Farama Foundation, "Gymnasium," 2024. [Online]. Available: https://gymnasium.farama.org/.

[18] OpenStreetMap Foundation, "El-Tahrir Square, Cairo," 2024. [Online]. Available: https://www.openstreetmap.org/#map=18/30.044309/31.235622.

[19] Giza Systems, "Intelligent transportation solutions (NAC)," 2024. [Online]. Available: https://www.gizasystems.com/solutions/transportation/.

[20] Gulf News, "Dubai RTA launches next-generation AI-powered traffic signal system," 2024. [Online]. Available: https://gulfnews.com/uae/transport/dubai-rta-launches-next-generation-ai-powered-traffic-signal-system-1.500195097.

[21] R. Waleed, R. Raafat, and M. Alhaj, "SUMOFlow AI — Traffic Optimization," GitHub, 2024. [Online]. Available: https://github.com/ranna-waleed/sumoflow-ai-traffic-optimization.

[22] OWASP Foundation, "OWASP Top Ten," 2025. [Online]. Available: https://owasp.org/www-project-top-ten/.

Appendix A: User Guide

A.1 System Requirements

Component	Minimum Requirement
Operating System	Ubuntu 20.04 LTS or Windows 10 (WSL2 recommended)
Python	3.9 or later (project uses 3.12)
Node.js	18 or later
GPU (optional)	NVIDIA GPU with CUDA 11.8+ for real-time YOLO inference
RAM	8 GB minimum; 16 GB recommended for DQN training
SUMO	Version 1.18 or later with sumo-gui installed
Disk space	~5 GB for simulation outputs, model weights, and dataset

A.2 Installation

A.2.1 Clone Repository

Open a terminal and run:

```
git clone
https://github.com/ranna-waleed/sumoflow-ai-traffic-optimization.git
cd sumoflow-ai-traffic-optimization
```

A.2.2 Python Dependencies

```
pip install -r requirements.txt
```

The main packages installed are fastapi, uvicorn, torch, torchvision, ultralytics, traci, sumolib, mlflow, dvc, pyyaml, numpy, pandas, scikit-learn, Pillow, and opencv-python.

A.2.3 Frontend Dependencies

```
cd frontend && npm install
```

A.2.4 Environment Variable

On Linux or macOS:

```
export SUMO_HOME=/usr/share/sumo    # Linux/macOS
# Add to ~/.bashrc for persistence
```

To make this permanent add the line above to your .bashrc or .zshrc file and restart the terminal.

A.3 Configuration

The DQN junction configuration is stored in backend/dqn_config.yaml. This file lists all seven junction IDs, their incoming lanes, phase mappings, and training hyperparameters. To adapt the system to a different intersection, edit this file with the new junction IDs and lane lists. No Python code changes are needed.

A.4 Running the System

A.4.1 Start Backend

```
cd backend
uvicorn main:app --reload --port 8000
```

Once running, full API documentation is available at http://localhost:8000/docs. Every endpoint can be tested directly from this page.

A.4.2 Start Frontend

```
cd frontend && npm start
```

The application opens automatically at http://localhost:3000. The Online badge in the top bar confirms the backend is connected. If the backend is not running the application switches to video mode automatically and plays pre-recorded footage.

A.5 Running a Simulation

Step 1: Open the application at http://localhost:3000 and go to the Dashboard page using the bottom navigation bar.

Step 2: Select a traffic profile from the four options: Morning Rush, Evening Rush, Midday, or Night.

Step 3: Click Start Simulation. The SUMO-GUI window opens and the simulation viewport begins showing live frames.

Step 4: Watch the four KPI cards update every two seconds showing vehicle count, average wait, CO2 emissions, and maximum individual wait. The traffic signal panel on the right shows the current phase for each of the seven junctions.

Step 5: Click Stop Simulation when finished. All metrics are saved to a CSV file automatically.

A.6 Running the DQN Comparison

Step 1: Go to the Before vs After page using the bottom navigation bar.

Step 2: Select a traffic profile.

Step 3: Click Run DQN Simulation. A second SUMO instance starts on a separate port so it does not interfere with any running standard simulation.

Step 4: Watch the junction decision panel on the right, which shows SWITCH or KEEP for each junction updated every 500 milliseconds.

Step 5: The comparison charts at the bottom update live showing DQN waiting time and CO2 against the fixed-time baseline.

A.7 API Usage Examples

A.7.1 Single Image Detection

```
curl -X http://localhost:8000/api/detection/detect  \
     -F 'file=@frame.jpg' \
     -F 'confidence=0.5'
```

A.7.2 Get Model Comparison

curl http://localhost:8000/api/models/comparison

A.7.3 Start Morning Rush Simulation

```
curl -X POST http://localhost:8000/api/sumo/start \
     -H "Content-Type: application/json" \
     -d '{"profile":"morning_rush","gui":true}'
```

A.8 Deployment

The frontend is deployed and publicly accessible on Netlify at:

https://cute-entremet-5bea9e.netlify.app/

No installation is needed to view the frontend. Simply open the link in any browser. Since the backend requires SUMO to be running locally, the deployed version automatically runs in video mode, playing pre-recorded simulation footage and showing static data. This allows anyone to explore the full interface and all five pages without needing to set up anything on their machine.

To run the system in live mode with a real simulation, the backend must be running locally on the same machine as SUMO. Follow the installation steps in sections A.2 to A.4 to set this up, then open http://localhost:3000 instead of the Netlify link.

A.9 Troubleshooting

Symptom	Likely Cause	Resolution
SUMO-GUI black window	View not positioned before screenshot	Wait 5 to 10 seconds. The service retries automatically up to 10 times

Frontend shows Offline	Backend not running	Start uvicorn and check that port 8000 is accessible. Check the browser console for errors
BiLSTM shows 'collecting'	Fewer than 60 steps collected	This is normal during the first 10 minutes of any simulation. No action needed
DQN Q-values all zero	Checkpoint not found	Check that checkpoint files exist in the backend/checkpoints/ folder
Port 8813 in use	Previous SUMO still running	Run pkill sumo-gui in the terminal before restarting the backend

Appendix B: Supplementary Materials

B.1 GitHub Repository

Repository: https://github.com/ranna-waleed/sumoflow-ai-traffic-optimization [21]

Directory / File	Contents
simulation/	SUMO network files, route generator, 5 profile configs, emission outputs
detection/	Frame extractor, auto-labeller, dataset splitter, 3 training scripts (YOLO/FRCNN/RetinaNet)
lstm/	prepare_data.py, train_lstm.py, predict.py, evaluate.py, scaler.pkl, config.json
DeepQN/	DuelingQNetwork, DQNAgent, MultiAgentDQN, 7-agent trainer, checkpoints/, configs/
DeepQN/tests/	test_ai_behavior.py containing 12 edge-case and adversarial tests
backend/	FastAPI app: main.py, 6 routers, 4 services, dqn_config.yaml, requirements.txt
frontend/	React 18: 5 pages, 2 hooks, 4 components, Tailwind CSS, Recharts
mlflow.db	MLflow experiment tracking SQLite database (all 3 model runs)
dataset.dvc	DVC tracking: 44 MB dataset, 1,801 files, MD5 hash verified
README.md	Setup, deployment, usage, and troubleshooting guide

Table B.1: GitHub repository structure summary

B.2 Dataset Description

Property	Value
Total images	1,801 PNG frames (1920×1080)
Label formats	YOLO .txt (for YOLOv8s); Pascal VOC .xml (for Faster R-CNN, RetinaNet)
Vehicle classes	7: car, bus, truck, motorcycle, taxi, microbus, bicycle
Split	Train 1,260 images (70%), Validation 361 images (20%), Test 180 images (10%)
Random seed	42 (fully reproducible)
Total size	44 MB (tracked by DVC with MD5 hash)

Labelling method	Automated colour-segmentation with zero manual annotation
Simulation source	SUMO 1.24, Tahrir Square morning rush, 3,600 steps

Table B.2: Training dataset description

B.3 Detailed Use Cases

Use Case	Actor	Precondition	Main Flow	Expected Result
Launch simulation	Traffic engineer	Backend online, profile selected	Click Start ->SUMO-GUI opens ->frames stream	Live KPIs update every 2s
Compare DQN vs baseline	Traffic engineer	Baseline CSV exists, DQN checkpoints loaded	Select profile on BeforeAfter ->Run DQN	Side-by-side metrics + learning curve
Detect vehicles in image	Developer	YOLO weights (best.pt) loaded	POST image to /detect with confidence=0.5	JSON with bounding boxes and class labels
Predict traffic flow	System (internal)	60 or more steps of history accumulated	BiLSTM predict() called every 90 steps	30-step N/S/E/W forecast in under 10 milliseconds
Adapt to new intersection	Developer	New OSM map and YAML config	Edit dqn_config.yaml; run netconvert; retrain	New agents trained on new topology

Table B.3: Detailed system use cases

B.4 Feasibility Study Summary

Three dimensions of feasibility were assessed for real-world deployment of SUMOFlow AI.

Technical feasibility: all components including SUMO, YOLOv8s, BiLSTM, DQN, FastAPI, and React are open-source and actively maintained. The seven DQN agent networks together contain around 987,000 parameters and can run inference in under two seconds on CPU, well within the ten-second decision interval and without requiring a GPU.

Economic feasibility: the frontend is already deployed for free on Netlify. The only operational cost for a real deployment would be a server to run the backend and SUMO simulation, which can be a standard cloud instance.

Organisational feasibility: adding a new intersection only requires editing the YAML configuration file. The React frontend needs no specialist software to operate, which means non-engineer traffic authority staff can use it directly without training.

B.5 Additional Evaluation Results

Profile	Avg Wait Reduction	CO2 Reduction	Throughput Change
Morning Rush	−86.5% (627s ->84s)	−58.6% (4,581M ->1,897M mg)	-97 vehicles
Evening Rush	−17.4% (632s ->523s)	−36.3% (7,460M ->4,746M mg)	+138 vehicles
Midday	+17.3% worse (629s ->738s)	+14.2% worse (4,109M ->4,694M mg)	-61 vehicles
Night	−14.2% (24s ->21s)	+9.1% worse (136M ->148M mg)	0 vehicles

Table B.4: Complete DQN evaluation results across all four profiles

B.6 Technology Stack Reference

Technology	Version	Role
Eclipse SUMO + TraCI	1.24	Microscopic traffic simulation and real-time control
YOLOv8s (Ultralytics)	8.4.18	Real-time vehicle detection at 208 fps on T4 GPU
Faster R-CNN	torchvision 0.15	Two-stage detector (comparison baseline)
RetinaNet	torchvision 0.15	Single-stage detector with Focal Loss (comparison)
PyTorch	2.12	Deep learning framework for all neural networks
BiLSTM (custom)	Not applicable	30-second directional traffic flow prediction
Dueling Double DQN (custom)	Not applicable	7 adaptive signal control agents
FastAPI	0.111	REST API backend with automatic OpenAPI docs
Uvicorn	0.29	ASGI server for FastAPI

React	18	Interactive web frontend framework
Recharts	latest	Charting library for metrics visualisation
Tailwind CSS	3	Utility-first CSS framework
MLflow	2.x	Experiment tracking and model registry
DVC	3.x	Dataset and model version control with MD5 hashing
OpenCV	4.x	Image processing for automated dataset labelling
OpenStreetMap	ODbL	Road network source data

Table B.5: Complete technology stack reference